Roteiro de Estudo 1

MySQL Básico: SELECT e SELECT com WHERE

Prof. Márcio Fantini
Colégio Técnico COLTEC – UFMG

8 de agosto de 2026

1 Introdução

Neste roteiro revisamos os comandos básicos de consulta em SQL: o comando `SELECT`, usado para recuperar dados de uma tabela, e a cláusula `WHERE`, usada para filtrar linhas de acordo com uma condição lógica.

Usaremos como banco de dados de exemplo o banco *livraria01*, que possui três tabelas: `editora`, `assunto` e `livro`. O script SQL que cria e popula esse banco (`livraria01.sql`) é apresentado na Seção 5 deste roteiro.

Código	Nome
1	Mirandela Editora
2	Editora Via-Norte
3	Editora Ilhas Tijucas
4	Maria José Editora

Tabela 1: Banco de Dados Livraria01 – Tabela Editora.

Sigla	Descrição
B	Banco de Dados
P	Programação
R	Redes
S	Sistemas Operacionais

Tabela 2: Banco de Dados Livraria01 – Tabela Assunto.

Código	Título	Preço	Lançamento	Assunto/Editora
1	Banco de Dados para Web	31,20	10/01/1999	B / 1
2	Programando em Linguagem C	30,00	01/10/1997	P / 1
3	Programando em Linguagem C++	111,50	01/11/1998	P / 3
4	Banco de Dados na Bioinformática	48,00	—	B / 2
5	Redes de Computadores	42,00	01/09/1996	R / 2

Tabela 3: Banco de Dados Livraria01 – Tabela Livro.

2 Usando o banco de dados: USE e SHOW TABLES

Exemplo

```
mysql> USE livraria01;
Database changed

mysql> SHOW TABLES;
+----------------------+
| Tables_in_livraria01 |
+----------------------+
| assunto              |
| editora              |
| livro                |
+----------------------+
3 rows in set (0.02 sec)

mysql> SELECT * FROM livro;
+--------+-----------------------------------+--------+------------+---------+---------+
| codigo | titulo                            | preco  | lancamento | assunto | editora |
+--------+-----------------------------------+--------+------------+---------+---------+
| 1      | Banco de Dados para Web           | 31.20  | 1999-01-10 | B       | 1       |
| 2      | Programando em Linguagem C        | 30.00  | 1997-10-01 | P       | 1       |
| 3      | Programando em Linguagem C++      | 111.50 | 1998-11-01 | P       | 3       |
| 4      | Banco de Dados na Bioinformatica  | 48.00  | NULL       | B       | 2       |
| 5      | Redes de Computadores             | 42.00  | 1996-09-01 | R       | 2       |
+--------+-----------------------------------+--------+------------+---------+---------+
5 rows in set (0.00 sec)
```

3 SELECT com WHERE

Definição

A cláusula `WHERE` permite filtrar as linhas retornadas por uma consulta, de acordo com uma condição lógica. Somente linhas para as quais a condição é verdadeira são incluídas no resultado.

Exemplo

Livros com preço maior que 30:

```
SELECT * FROM livro WHERE preco > 30;
```

Combinando condições com `AND`:

```
SELECT * FROM livro
WHERE preco > 30 AND assunto = 'B';
```

3.1 Testando valores nulos: IS NULL e IS NOT NULL

Dica

Para testar se uma coluna possui valor nulo, **não** se deve usar o operador de igualdade (`= NULL`), pois NULL representa ausência de valor e não é comparável por igualdade. Deve-se usar `IS NULL` ou `IS NOT NULL`.

Exemplo

```
SELECT * FROM livro WHERE lancamento IS NULL;

SELECT * FROM livro WHERE lancamento IS NOT NULL;
```

4 O operador LIKE

Definição

O operador `LIKE` é usado em conjunto com a cláusula `WHERE` para buscar um padrão de texto em uma coluna. Os curingas mais usados são:

- `%` – qualquer sequência de zero ou mais caracteres;
- `_` – exatamente um caractere qualquer.

Exemplo

```
-- Livros cujo titulo comeca com "BANCO"
SELECT * FROM livro WHERE titulo LIKE 'BANCO%';

-- Editoras cujo nome comeca com "edit" (LIKE nao diferencia maiusculas/minusculas por padrao)
SELECT * FROM editora WHERE nome LIKE 'edit%';

-- Livros cujo titulo contenha "LINGUAGEM C"
SELECT codigo, titulo, preco
FROM livro
WHERE titulo LIKE '%LINGUAGEM C%';
```

5 Banco de Dados Utilizado neste Roteiro

Banco de Dados Utilizado

Este roteiro usa o banco *livraria01*. O script completo (`CREATE DATABASE`, `CREATE TABLE` e `INSERT`) está no arquivo **livraria01.sql**, fornecido junto com este roteiro. Para carregá-lo:

```
mysql -u seu_usuario -p < livraria01.sql
```

6 Exercícios

Exercício

1. Liste os livros cujo assunto seja igual a ‘P’.
2. Liste os livros com preço entre 30 e 50 reais (use `AND`).
3. Liste as editoras cujo nome contenha a palavra “Editora”.
4. Liste os livros que ainda não possuem data de lançamento definida.

7 Solução

Solução

1. Livros cujo assunto seja ‘P’:

```
SELECT * FROM livro WHERE assunto = 'P';
```

2. Livros com preço entre 30 e 50 reais:

```
SELECT * FROM livro WHERE preco >= 30 AND preco <= 50;

-- Ou, de forma equivalente, usando BETWEEN:
SELECT * FROM livro WHERE preco BETWEEN 30 AND 50;
```

3. Editoras cujo nome contenha “Editora”:

```
SELECT * FROM editora WHERE nome LIKE '%Editora%';
```

4. Livros sem data de lançamento definida:

```
SELECT * FROM livro WHERE lancamento IS NULL;
```

Compilado em 8 de agosto de 2026 – 10:19:47.

Roteiro de Estudo 2

Funções de Agregação: COUNT, SUM, MAX e MIN

Prof. Márcio Fantini
Colégio Técnico COLTEC – UFMG

8 de agosto de 2026

1 Introdução

Em SQL existem funções que operam sobre *conjuntos* de dados: em vez de devolver um valor por linha, elas resumem várias linhas em um único resultado (uma relação de uma linha e uma coluna). Essas são as **funções agregadas**. Neste roteiro veremos as quatro mais comuns: `COUNT`, `SUM`, `MAX` e `MIN`. Continuamos usando o banco *livraria01* do Roteiro de Estudo 1.

2 COUNT – Contagem

Definição

A função `COUNT` conta o número de linhas que satisfazem uma condição. Como parâmetro, pode-se usar o símbolo `*` (conta todas as linhas) ou o nome de uma coluna (conta apenas as ocorrências *não-nulas* daquela coluna).

Exemplo

```sql
-- Quantidade total de linhas da tabela de livros
SELECT COUNT(*)
FROM livro;

-- Quantidade de livros com o codigo preenchido
SELECT COUNT(codigo)
FROM livro;

-- Quantidade de livros com data de lancamento preenchida
SELECT COUNT(lancamento)
FROM livro;
```

Dica

Repare a diferença entre `COUNT(*)` e `COUNT(coluna)`: o primeiro conta todas as linhas da tabela, enquanto o segundo ignora as linhas em que a coluna é `NULL`. No exemplo do livro sem data de lançamento, `COUNT(*)` retorna 5, mas `COUNT(lancamento)` retorna apenas 4.

3 SUM – Soma

Definição

A função `SUM` soma os valores (numéricos) de uma coluna para um conjunto de linhas.

Exemplo

```
-- Somatorio dos precos da tabela de livros
SELECT SUM(preco)
FROM livro;
```

4 AVG – Média

Definição

A função `AVG` calcula a média aritmética dos valores de uma coluna, considerando apenas os valores não nulos.

Exemplo

```
-- Media dos precos de livros
SELECT AVG(preco)
FROM livro;
```

5 MAX e MIN – Valores máximo e mínimo

Exemplo

```
-- Maior preco entre os livros de assunto 'P'
SELECT MAX(preco)
FROM livro
WHERE assunto = 'P';

-- Menor preco entre os livros de assunto 'B'
SELECT MIN(preco)
FROM livro
WHERE assunto = 'B';
```

6 Banco de Dados Utilizado neste Roteiro

Banco de Dados Utilizado

Este roteiro usa o mesmo banco *livraria01* do Roteiro de Estudo 1. O script completo está no arquivo **livraria01.sql**, fornecido junto com este roteiro.

```
mysql -u seu_usuario -p < livraria01.sql
```

7 Exercícios

Exercício

1. Quantos livros ao todo existem na tabela `livro`?
2. Qual o preço total de todos os livros do assunto 'B'?
3. Qual é o preço médio de todos os livros já lançados?
4. Qual é o livro mais caro e qual é o mais barato (mostre apenas o valor do preço)?

8 Solução

Solução

1. Quantidade total de livros:

```
SELECT COUNT(*) FROM livro;
```

2. Preço total dos livros do assunto 'B':

```
SELECT SUM(preco) FROM livro WHERE assunto = 'B';
```

3. Preço médio dos livros já lançados:

```
SELECT AVG(preco) FROM livro WHERE lancamento IS NOT NULL;
```

4. Preço do livro mais caro e do mais barato:

```
SELECT MAX(preco) AS mais_caro, MIN(preco) AS mais_barato
FROM livro;
```

Compilado em 8 de agosto de 2026 – 10:19:50.

Roteiro de Estudo 3

Outras Funções e Recursos Muito Usados em SELECT

Prof. Márcio Fantini
Colégio Técnico COLTEC – UFMG

8 de agosto de 2026

1 Introdução

Além das funções agregadas, o MySQL oferece diversos recursos que tornam as consultas mais legíveis e poderosas: apelidos, concatenação, operações aritméticas, remoção de duplicatas, ordenação, o predicado `IN`, funções de string, funções de data e funções matemáticas. Continuamos usando o banco *livraria01*.

2 Apelidos (aliases)

Definição

A palavra-chave `AS` permite atribuir um apelido a uma coluna ou a uma tabela resultante de uma consulta, tornando a saída mais legível ou simplificando referências em consultas mais complexas. Para tabelas, o `AS` é opcional.

Exemplo

```sql
-- Apelido para coluna
SELECT MAX(preco) AS preco_maximo
FROM livro
WHERE assunto = 'P';

-- Apelido para tabela (com notacao de ponto)
SELECT MAX(l.preco) AS preco_maximo
FROM livro AS l
WHERE assunto = 'P';

-- O AS para tabela e opcional
SELECT MAX(l.preco) AS preco_maximo
FROM livro l
WHERE assunto = 'P';
```

3 Concatenação: CONCAT()

Exemplo

```
SELECT CONCAT('LIVRO: ', titulo) AS texto
FROM livro;
```

4 Operações aritméticas

Exemplo

```
-- Novos precos reajustados em 10%
SELECT titulo, preco * 1.1 AS novo_preco
FROM livro;
```

5 DISTINCT

Definição

O predicado `DISTINCT` elimina linhas repetidas do resultado de uma consulta.

Exemplo

```
SELECT DISTINCT assunto AS assunto
FROM livro;
```

6 ORDER BY – Ordenando resultados

Definição

A cláusula `ORDER BY` é sempre a última de uma consulta e define a ordem de apresentação das linhas (`ASC` para crescente, `DESC` para decrescente).

Exemplo

```
-- Ordem crescente de assunto e decrescente de preco
SELECT assunto, titulo, preco
FROM livro
ORDER BY assunto, preco DESC;
```

7 O predicado IN

Definição

O predicado `IN` testa se um valor pertence a uma lista de valores, sendo uma alternativa mais compacta a uma sequência de condições unidas por `OR`.

Exemplo

```
SELECT *
FROM livro
WHERE assunto IN ('S', 'P', 'B');
```

8 Funções de string

Definição

Algumas funções úteis para manipular texto:

- `UPPER` / `LOWER`: converte para maiúsculas/minúsculas;
- `TRIM`: remove espaços das extremidades da string;
- `SUBSTRING(str, inicio, comprimento)`: extrai um trecho da string;
- `LENGTH`: retorna o comprimento da string.

Exemplo

```
SELECT SUBSTRING(titulo, 1, 10) AS titulo
FROM livro;

SELECT LENGTH(titulo) AS comprimento
FROM livro
WHERE assunto = 'B';

SELECT UPPER(titulo)
FROM livro
WHERE assunto = 'R';
```

9 Funções de data

Exemplo

```
SELECT titulo, DAY(lancamento) AS dia_lancamento
FROM livro;

SELECT titulo, MONTH(lancamento) AS mes, YEAR(lancamento) AS ano
FROM livro;
```

10 Funções aritméticas: FLOOR, CEIL, POWER, ROUND

Exemplo

```
SELECT codigo, CEIL(preco) FROM livro;
SELECT codigo, FLOOR(preco) FROM livro;
SELECT codigo, ROUND(POWER(preco, 2)) FROM livro;
```

Dica

A lista completa de funções matemáticas do MySQL pode ser consultada na documentação oficial: https://dev.mysql.com/doc/refman/8.0/en/mathematical-functions.html

11 Banco de Dados Utilizado neste Roteiro

Banco de Dados Utilizado

Este roteiro usa o mesmo banco *livraria01* dos roteiros anteriores. O script completo está no arquivo **livraria01.sql**, fornecido junto com este roteiro.

```
mysql -u seu_usuario -p < livraria01.sql
```

12 Exercícios

Exercício

1. Liste os nomes das editoras ordenados em ordem crescente, com a coluna nomeada como “Editora”.
2. Mostre título e preço dos livros, ordenados em ordem decrescente de preço.
3. Mostre código e preço dos livros e mais três colunas com o preço acrescido de 10%, 20% e 30%.
4. Liste os títulos dos livros com mais de 20 caracteres.

13 Solução

Solução

1. Nomes das editoras, ordem crescente:

```
SELECT nome AS Editora
FROM editora
ORDER BY nome ASC;
```

2. Título e preço, ordem decrescente de preço:

```
SELECT titulo, preco
FROM livro
ORDER BY preco DESC;
```

3. Código, preço e preço reajustado em 10%, 20% e 30%:

```
SELECT codigo, preco,
       preco * 1.1 AS Preco10,
       preco * 1.2 AS Preco20,
       preco * 1.3 AS Preco30
FROM livro;
```

4. Títulos com mais de 20 caracteres:

```
SELECT titulo
FROM livro
WHERE LENGTH(titulo) > 20;
```

Compilado em 8 de agosto de 2026 – 10:19:52.

Roteiro de Estudo 4

Funções de Agregação com GROUP BY e HAVING

Prof. Márcio Fantini
Colégio Técnico COLTEC – UFMG

8 de agosto de 2026

1 Introdução

As funções agregadas vistas no Roteiro de Estudo 2 transformam *todas* as linhas de uma tabela em um único resultado. Mas como calcular, por exemplo, o preço médio dos livros *separadamente* para cada assunto? Para isso usamos a cláusula `GROUP BY`, que organiza as linhas em grupos de acordo com os valores de uma ou mais colunas, aplicando a função agregada a cada grupo. Continuamos usando o banco *livraria01*.

Dica

A consulta abaixo **não** calcula o preço médio por assunto – ela calcula a média geral, ignorando o agrupamento:

```
SELECT assunto, AVG(preco)
FROM livro;
```

Sem o `GROUP BY`, o MySQL (em modo estrito) sequer permite misturar uma coluna comum com uma função agregada dessa forma.

2 Cláusula GROUP BY

Exemplo

```
-- Preco medio por assunto
SELECT assunto, AVG(preco)
FROM livro
GROUP BY assunto;

-- Quantidade de livros por assunto
SELECT assunto, COUNT(*)
FROM livro
GROUP BY assunto;

-- Preco do livro mais caro de cada assunto, entre os ja lancados
SELECT assunto, MAX(preco)
FROM livro
WHERE lancamento IS NOT NULL
GROUP BY assunto;

-- Quantos livros ja foram lancados por cada editora
SELECT editora, COUNT(*)
FROM livro
WHERE lancamento IS NOT NULL
GROUP BY editora;
```

```sql
-- Media de preco para cada combinacao de codigo e assunto
SELECT codigo, assunto, AVG(preco)
FROM livro
GROUP BY codigo, assunto;
```

Dica

Para cada valor distinto da combinação das colunas especificadas no `GROUP BY`, será gerada uma linha no resultado da consulta.

3 Cláusula HAVING

Definição

A cláusula `WHERE` filtra linhas *antes* de elas serem agrupadas – por isso não pode usar funções agregadas em sua condição. A cláusula `HAVING` filtra os *grupos já formados*, aplicada *depois* do `GROUP BY` e do cálculo das funções agregadas. Por isso é na `HAVING` que se usam condições com funções agregadas.

Exemplo

Sintaxe básica:

```sql
SELECT col1, col2, ..., funcao_agregada
FROM nome_tabela
WHERE expressao_logica
GROUP BY col1, col2, ...
HAVING expressao_logica_com_agregada;
```

```sql
-- Assuntos cujo preco medio dos livros ultrapassa R$ 50,00
SELECT assunto
FROM livro
GROUP BY assunto
HAVING AVG(preco) > 50;

-- Assuntos que possuem pelo menos dois livros
SELECT assunto, COUNT(*)
FROM livro
GROUP BY assunto
HAVING COUNT(*) > 1;

-- Assuntos com pelo menos dois livros ja lancados
SELECT assunto, COUNT(*)
FROM livro
WHERE lancamento IS NOT NULL
GROUP BY assunto
HAVING COUNT(*) > 1;
```

4 Banco de Dados Utilizado neste Roteiro

Banco de Dados Utilizado

Este roteiro usa o mesmo banco *livraria01* dos roteiros anteriores. O script completo está no arquivo **livraria01.sql**, fornecido junto com este roteiro.

```
mysql -u seu_usuario -p < livraria01.sql
```

5 Exercícios

Exercício

1. Para cada editora, mostre a quantidade de livros e o preço médio.
2. Liste apenas os assuntos cuja soma dos preços dos livros ultrapasse R$ 70,00.
3. Liste as editoras que possuem mais de um livro cadastrado.
4. Combine `WHERE` e `HAVING` numa mesma consulta: considere apenas livros com preço maior que 25 e depois filtre os assuntos com pelo menos dois livros nessas condições.

6 Solução

Solução

1. Quantidade de livros e preço médio por editora:

```
SELECT editora, COUNT(*) AS qtd_livros, AVG(preco) AS preco_medio
FROM livro
GROUP BY editora;
```

2. Assuntos cuja soma dos preços ultrapasse R$ 70,00:

```
SELECT assunto, SUM(preco) AS soma_precos
FROM livro
GROUP BY assunto
HAVING SUM(preco) > 70;
```

3. Editoras com mais de um livro cadastrado:

```
SELECT editora, COUNT(*) AS qtd_livros
FROM livro
GROUP BY editora
HAVING COUNT(*) > 1;
```

4. Livros com preço maior que 25, assuntos com pelo menos dois livros nessas condições:

```
SELECT assunto, COUNT(*) AS qtd_livros
FROM livro
WHERE preco > 25
GROUP BY assunto
HAVING COUNT(*) >= 2;
```

Compilado em 8 de agosto de 2026 – 10:19:54.

Roteiro de Estudo 5

Consultas Intermediárias: Subconsultas, Conjuntos, Views, CTEs e Transações

Prof. Márcio Fantini
Colégio Técnico COLTEC – UFMG

8 de agosto de 2026

1 Introdução

Neste roteiro avançamos para tópicos intermediários: subconsultas, operações de conjunto, views, CTEs e transações. Cada seção usa um banco de dados de exemplo diferente, todos fornecidos junto com este roteiro (ver Seção 6).

2 Subconsultas (subqueries)

Definição

Uma **subconsulta** é uma consulta aninhada dentro de outra consulta, podendo ser usada com `SELECT`, `INSERT`, `UPDATE` ou `DELETE`. A consulta que contém a subconsulta é chamada de *consulta externa*, e a subconsulta é chamada de *consulta interna*. Uma subconsulta pode ser usada em qualquer lugar em que uma expressão é permitida, e deve estar entre parênteses.

Nesta seção usamos o banco de dados *classicmodels*, um banco clássico de exemplo com dados comerciais (clientes, produtos, pedidos, pagamentos etc.), amplamente usado no ensino de bancos de dados.

Exemplo

Funcionários que trabalham em escritórios localizados nos EUA:

```sql
SELECT lastName, firstName
FROM employees
WHERE officeCode IN (
    SELECT officeCode
    FROM offices
    WHERE country = 'USA'
);
```

2.1 Subconsultas não-relacionadas e relacionadas

Definição

As subconsultas podem ser classificadas em:

- **Não-relacionadas (independentes)**: podem ser executadas sozinhas, sem depender da consulta externa. Usadas com operadores de comparação, `IN`/`NOT IN`, ou como

tabela derivada na cláusula `FROM`.

- **Relacionadas (correlacionadas)**: dependem de valores da consulta externa e são reavaliadas para cada linha dela. Usadas tipicamente com `EXISTS` e `NOT EXISTS`.

2.1.1 WHERE com operadores de comparação

Exemplo

Cliente que fez o pagamento mais alto:

```
SELECT customerNumber, checkNumber, amount
FROM payments
WHERE amount = (SELECT MAX(amount) FROM payments);
```

Clientes cujos pagamentos são maiores que o pagamento médio:

```
SELECT customerNumber, checkNumber, amount
FROM payments
WHERE amount > (SELECT AVG(amount) FROM payments);
```

2.1.2 IN e NOT IN

Exemplo

Clientes que não fizeram nenhum pedido:

```
SELECT customerName
FROM customers
WHERE customerNumber NOT IN (
    SELECT DISTINCT customerNumber FROM orders
);
```

2.1.3 Subconsulta na cláusula FROM (tabela derivada)

Exemplo

Número máximo, mínimo e médio de itens por pedido de venda:

```
SELECT MAX(items), MIN(items), FLOOR(AVG(items))
FROM (
    SELECT orderNumber, COUNT(orderNumber) AS items
    FROM orderdetails
    GROUP BY orderNumber
) AS lineitems;
```

2.1.4 Subconsulta correlacionada

Exemplo

Produtos cujo preço de compra é maior que o preço médio de compra *da mesma linha de produto*:

```
SELECT p1.productname, p1.buyprice
FROM products p1
WHERE p1.buyprice > (
    SELECT AVG(buyprice)
    FROM products
    WHERE productline = p1.productline
);
```

Dica

Diferente de uma subconsulta independente, uma subconsulta correlacionada não pode ser executada isoladamente: ela depende do apelido da tabela externa (`p1` no exemplo), que só existe no contexto da consulta completa.

2.1.5 EXISTS e NOT EXISTS

Definição

`EXISTS` testa se a subconsulta retorna alguma linha (verdadeiro) ou nenhuma (falso), sendo muito usado com subconsultas correlacionadas.

Exemplo

Clientes que fizeram pelo menos um pedido com valor total superior a 60 mil:

```
SELECT customerNumber, customerName
FROM customers
WHERE EXISTS (
    SELECT orderNumber, SUM(priceEach * quantityOrdered)
    FROM orderdetails
    INNER JOIN orders USING (orderNumber)
    WHERE customerNumber = customers.customerNumber
    GROUP BY orderNumber
    HAVING SUM(priceEach * quantityOrdered) > 60000
);
```

3 Operações de conjunto: UNION, INTERSECT e diferença

Nesta seção usamos o banco de dados *conjuntos*, criado especificamente para ilustrar junções e operações de conjunto, com as tabelas `t1`, `t2`, `tab1` e `tab2`.

3.1 Junção simples vs. produto cartesiano

Dica

Selecionar colunas de duas tabelas sem uma condição de junção resulta em um **produto cartesiano** (cada linha de uma tabela combinada com cada linha da outra) – quase nunca é o que se deseja.

Exemplo

```
-- Produto cartesiano (evitar!)
SELECT t1.id id1, t1.valor valor1, t2.id id2, t2.valor valor2
FROM tab1 t1, tab2 t2;

-- Juncao correta, usando INNER JOIN ... USING
SELECT t1.id id1, t1.valor valor1, t2.id id2, t2.valor valor2
FROM tab1 t1
INNER JOIN tab2 t2 USING (id);
```

3.2 UNION e UNION ALL

Definição

O predicado `UNION` combina os resultados de duas consultas, removendo linhas duplicadas. `UNION ALL` faz o mesmo, mas mantém todas as duplicatas. Para usar `UNION`, as consultas devem retornar o mesmo número de colunas, com tipos compatíveis.

Exemplo

```
SELECT * FROM tab1
UNION
SELECT * FROM tab2;

SELECT * FROM tab1
UNION ALL
SELECT * FROM tab2;
```

3.3 Interseção

Definição

O MySQL não possui o operador `INTERSECT` (em versões mais antigas); a interseção entre dois conjuntos pode ser obtida por meio de uma junção com `DISTINCT`.

Exemplo

Interseção entre as tabelas `t1` e `t2`:

```
SELECT DISTINCT id
FROM t1
INNER JOIN t2 USING (id);
```

Dica

O uso de `DISTINCT` é fundamental aqui: sem ele, valores repetidos em uma das tabelas geram linhas duplicadas no resultado da junção.

3.4 Diferença: NOT IN

Definição

As cláusulas `EXCEPT` (PostgreSQL) e `MINUS` (Oracle/SQL Server) calculam a diferença entre dois conjuntos. No MySQL tradicional, usa-se `NOT IN` com uma subconsulta para obter o mesmo resultado.

Exemplo

```sql
-- Elementos de conj1 que NAO estao em conj2
SELECT numero FROM conj1
WHERE numero NOT IN (SELECT numero FROM conj2);

-- Elementos de conj2 que NAO estao em conj1
SELECT numero FROM conj2
WHERE numero NOT IN (SELECT numero FROM conj1);
```

Dica

Nas operações de conjunto, as colunas envolvidas precisam ter o *mesmo tipo de dado*, mas não precisam ter o mesmo nome.

4 Views e CTEs

Nesta seção usamos o banco de dados *Aplicativos*, com as tabelas `Apps` (aplicativos da loja em geral) e `Google` (aplicativos da Google Play), ambas com colunas `category` e `rating`.

4.1 Subconsulta como tabela derivada

Exemplo

Comparando o rating de cada aplicativo Google com o rating médio da categoria correspondente, calculado por subconsulta:

```sql
SELECT ga.name, ga.category, ga.rating AS rating_google, app.rating_medio
FROM Google ga
INNER JOIN (
    SELECT category, ROUND(AVG(rating), 2) AS rating_medio
    FROM Apps
    GROUP BY category
) app ON ga.category = app.category;
```

4.2 VIEW

Definição

Uma `VIEW` é um objeto armazenado no banco de dados que representa o resultado de uma consulta, funcionando como uma "tabela virtual". A view não armazena os dados – apenas a definição da consulta; os dados são recalculados toda vez que a view é referenciada.

Exemplo

```sql
CREATE VIEW apps_medio AS
SELECT category, ROUND(AVG(rating), 3) AS med_rating
FROM Apps
GROUP BY category;

-- Usando a view em outra consulta
SELECT ga.name, ga.category, ga.rating, tam.med_rating
FROM Google ga
JOIN apps_medio tam ON ga.category = tam.category;
```

4.3 CTE (Common Table Expression)

Definição

Uma `CTE` é um resultado temporário, nomeado, criado com a palavra-chave `WITH`, e que só pode ser usado *imediatamente* na consulta em que foi definido – diferente da view, que persiste no banco.

Exemplo

```sql
WITH apps_medio AS (
    SELECT category, ROUND(AVG(rating), 2) AS medio_rating
    FROM Apps
    GROUP BY category
)
SELECT ga.name, ga.category, ga.rating, tam.medio_rating
FROM Google ga
JOIN apps_medio tam ON ga.category = tam.category;
```

Dica

VIEW vs. CTE: use uma CTE para consultas usadas apenas uma vez ou ocasionalmente; use uma VIEW quando a mesma consulta for reutilizada com frequência, ou quando for necessário restringir o acesso de determinados usuários a apenas parte dos dados do banco.

5 Transações: START TRANSACTION, COMMIT e ROLLBACK

Nesta seção usamos o banco *banks*, com as tabelas `accounts` (contas correntes) e `transactions` (movimentações).

Definição

Uma **transação** é uma sequência de instruções SQL executadas como uma única unidade de trabalho: ou todas as operações são aplicadas, ou nenhuma é (garantindo a integridade dos dados). O MySQL controla transações com três instruções principais:

- `START TRANSACTION` (ou `BEGIN`) – inicia a transação;
- `COMMIT` – confirma e aplica as alterações permanentemente;
- `ROLLBACK` – desfaz as alterações, retornando ao estado anterior ao início da transação.

Por padrão, o MySQL está em modo `autocommit`: cada instrução é automaticamente confirmada. Para controlar transações manualmente, desativa-se esse modo com `SET autocommit = 0;`.

Exemplo

Exemplo com `COMMIT`:

```
START TRANSACTION;

INSERT INTO users (id, username) VALUES (1, 'john');

UPDATE users
SET email = 'john.doe@example.com'
WHERE id = 1;

-- As alteracoes so ficam visiveis para outras sessoes apos o COMMIT
COMMIT;
```

Exemplo com `ROLLBACK`:

```
START TRANSACTION;

INSERT INTO users (id, username) VALUES (2, 'jane');

UPDATE users
SET email = 'jane.doe@example.com'
WHERE id = 2;

-- Desfaz TODAS as alteracoes feitas desde o START TRANSACTION
ROLLBACK;
```

5.1 Transações em Stored Procedures

Exemplo

Transferência de saldo entre duas contas, com `ROLLBACK` automático em caso de saldo insuficiente (a stored procedure já está pronta no script `banks.sql`):

```
CALL transfer(1, 2, 100);
```

Dica

Se a conta de origem não tiver saldo suficiente, a atualização deixa o saldo negativo, a condição do `IF` dentro da procedure é satisfeita, a transação inteira é desfeita com `ROLLBACK` e um erro é sinalizado ao cliente – os saldos das duas contas voltam exatamente ao estado

anterior à chamada.

6 Bancos de Dados Utilizados neste Roteiro

Banco de Dados Utilizado

Este roteiro usa **quatro** bancos de dados diferentes, um por seção. Os scripts SQL completos são fornecidos junto com este roteiro:

- **classicmodels_full.sql** – banco *classicmodels* (subconsultas). Este é o dump oficial e completo do banco de exemplo, com todas as tabelas (`customers`, `employees`, `offices`, `orders`, `orderdetails`, `payments`, `products`, `productlines`);
- **conjuntos.sql** – banco *conjuntos* (operações de conjunto);
- **aplicativos.sql** – banco *Aplicativos* (views e CTEs);
- **banks.sql** – banco *banks* (transações), já incluindo a stored procedure `transfer`.

Para carregar qualquer um deles:

```
mysql -u seu_usuario -p < nome_do_arquivo.sql
```

7 Exercícios

Exercício

1. No banco *classicmodels*, escreva uma subconsulta que liste os produtos cujo preço de venda (`MSRP`) é maior que o preço médio de venda de todos os produtos.
2. No banco *conjuntos*, obtenha a união (sem duplicatas) e a interseção entre `tab1` e `tab2`.
3. No banco *Aplicativos*, crie uma CTE que calcule o total de `reviews` por categoria e a combine com a tabela `Google`.
4. No banco *banks*, simule uma transferência que falharia por saldo insuficiente e verifique que os saldos permanecem inalterados após o `ROLLBACK`.

8 Solução

Solução

1. Produtos com `MSRP` acima da média:

```
SELECT productName, MSRP
FROM products
WHERE MSRP > (SELECT AVG(MSRP) FROM products);
```

2. União e interseção entre `tab1` e `tab2`:

```
-- Uniao (sem duplicatas)
SELECT * FROM tab1
```

```
UNION
SELECT * FROM tab2;

-- Intersecao (junção com DISTINCT, comparando pelo id)
SELECT DISTINCT t1.id, t1.valor
FROM tab1 t1
INNER JOIN tab2 t2 ON t1.id = t2.id;
```

3. CTE com total de `reviews` por categoria, combinada com `Google`:

```
WITH review_totais AS (
    SELECT category, SUM(reviews) AS total_reviews
    FROM Apps
    GROUP BY category
)
SELECT ga.name, ga.category, ga.reviews, rt.total_reviews
FROM Google ga
JOIN review_totais rt ON ga.category = rt.category;
```

4. Transferência que falha por saldo insuficiente:

```
-- Antes da chamada
SELECT * FROM accounts;

-- Tenta transferir um valor maior que o saldo disponivel
CALL transfer(1, 2, 999999.00);
-- ERROR 1644 (45000): Transaction rolled back: Insufficient funds

-- Confirma que os saldos NAO foram alterados
SELECT * FROM accounts;
```

Compilado em 8 de agosto de 2026 – 10:19:57.

Roteiro de Estudo 6

Bancos de Dados Usuais na Literatura de Ensino de Bancos de Dados

Prof. Márcio Fantini
Colégio Técnico COLTEC – UFMG

8 de agosto de 2026

1 Introdução

Um aspecto importante do aprendizado de SQL é praticar sobre bancos de dados de exemplo bem conhecidos, com dados realistas e volume suficiente para tornar as consultas interessantes. Nos roteiros anteriores já usamos vários desses bancos:

- **livraria01** – pequeno banco didático (editoras, assuntos, livros), útil para os primeiros contatos com `SELECT` e `WHERE`;
- **classicmodels** – banco clássico de dados comerciais (clientes, produtos, pedidos, pagamentos, funcionários, escritórios), amplamente usado para ensinar junções e subconsultas;
- **conjuntos** – banco criado especificamente para ilustrar junções e operações de conjunto;
- **Aplicativos** – banco com dados de aplicativos de duas lojas (`Apps` e `Google`), usado para VIEWs e CTEs;
- **banks** – banco simples de contas correntes, usado para ilustrar transações.

Neste roteiro acrescentamos um sexto banco de dados amplamente usado na literatura e em cursos de banco de dados: o **sakila**, banco de exemplo oficial do MySQL que simula uma locadora de filmes (DVD rental).

2 O banco de dados sakila

Definição

O banco *sakila* modela uma locadora de filmes fictícia. Entre suas principais tabelas estão:

- `film` – filmes (título, ano, duração, classificação, taxa de aluguel);
- `actor` – atores;
- `film_actor` – tabela associativa entre filmes e atores;
- `category` e `film_category` – categorias dos filmes;
- `customer` – clientes da locadora;
- `rental` – registros de aluguel de filmes;
- `payment` – pagamentos realizados pelos clientes;

- `store` – lojas da locadora.

É um banco maior e mais rico em relacionamentos que os anteriores, sendo muito usado para praticar junções múltiplas, agregações e subconsultas mais elaboradas.

Exemplo

Quantidade de filmes por categoria, em ordem decrescente:

```
SELECT c.name AS categoria, COUNT(*) AS qtd_filmes
FROM film_category fc
JOIN category c ON fc.category_id = c.category_id
GROUP BY c.name
ORDER BY qtd_filmes DESC;
```

Os 5 clientes que mais gastaram em pagamentos:

```
SELECT cu.customer_id, cu.first_name, cu.last_name,
       SUM(p.amount) AS total_pago
FROM customer cu
JOIN payment p ON cu.customer_id = p.customer_id
GROUP BY cu.customer_id, cu.first_name, cu.last_name
ORDER BY total_pago DESC
LIMIT 5;
```

Filmes com duração acima da média de duração de todos os filmes (subconsulta não-relacionada):

```
SELECT title, length
FROM film
WHERE length > (SELECT AVG(length) FROM film)
ORDER BY length DESC;
```

Atores que participaram de mais de 20 filmes (GROUP BY e HAVING):

```
SELECT a.actor_id, a.first_name, a.last_name, COUNT(*) AS qtd_filmes
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name
HAVING COUNT(*) > 20
ORDER BY qtd_filmes DESC;
```

3 Banco de Dados Utilizado neste Roteiro

Banco de Dados Utilizado

Este roteiro usa o banco oficial *sakila*, fornecido em dois arquivos junto com este roteiro:

- **sakila_schema.sql** – cria toda a estrutura (tabelas, views, stored procedures, triggers);
- **sakila_data.sql** – popula as tabelas com os dados oficiais da locadora fictícia.

Para carregar (nesta ordem):

```
mysql -u seu_usuario -p < sakila_schema.sql
mysql -u seu_usuario -p < sakila_data.sql
```

4 Exercícios

Exercício

1. No banco *sakila*, liste as 10 categorias de filme com maior taxa de aluguel média (`rental_rate`).
2. Liste os clientes que nunca fizeram nenhum aluguel (use `NOT EXISTS` ou `NOT IN`).
3. Compare, para o banco *classicmodels* e para o *sakila*, como uma mesma pergunta – "quem são os 5 melhores clientes em valor total pago?" – é respondida em cada um dos dois bancos.

5 Solução

Solução

1. 10 categorias com maior taxa de aluguel média:

```
SELECT c.name AS categoria, ROUND(AVG(f.rental_rate), 2) AS taxa_media
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY c.name
ORDER BY taxa_media DESC
LIMIT 10;
```

2. Clientes que nunca alugaram nenhum filme:

```
SELECT c.customer_id, c.first_name, c.last_name
FROM customer c
WHERE NOT EXISTS (
    SELECT 1
    FROM rental r
    WHERE r.customer_id = c.customer_id
);
```

3. Comparação entre os dois bancos:

```
-- classicmodels: 5 clientes com maior soma de pagamentos
SELECT customerName, SUM(amount) AS total_pago
FROM customers
JOIN payments USING (customerNumber)
GROUP BY customerName
ORDER BY total_pago DESC
LIMIT 5;

-- sakila: 5 clientes com maior soma de pagamentos
SELECT CONCAT(first_name, ' ', last_name) AS cliente,
       SUM(amount) AS total_pago
FROM customer
JOIN payment USING (customer_id)
GROUP BY cliente
ORDER BY total_pago DESC
LIMIT 5;
```

Em ambos os casos a estrutura da consulta é a mesma (junção entre cliente e pagamentos, agregação por `SUM`, ordenação decrescente e `LIMIT`); a diferença está apenas nos nomes das tabelas e colunas de cada modelo de dados.

Compilado em 8 de agosto de 2026 – 10:20:00.

Roteiro de Estudo 7

Consultas no Banco de Dados World

Prof. Márcio Fantini
Colégio Técnico COLTEC – UFMG

8 de agosto de 2026

1 Introdução

O banco de dados **world** é outro exemplo clássico, distribuído oficialmente pelo MySQL, que reúne dados geográficos e demográficos de países, cidades e idiomas do mundo todo. É um excelente banco para consolidar os conceitos vistos nos roteiros anteriores, pois combina tabelas de tamanho médio com relacionamentos simples e diretos.

Definição

O banco *world* possui três tabelas principais:

- `country` – países (código, nome, continente, região, população, área, PIB, capital – `Capital`, que referencia `City.ID`);
- `city` – cidades (nome, código do país – `CountryCode`, distrito, população);
- `countrylanguage` – idiomas falados em cada país (`CountryCode`, `Language`, se é oficial – `IsOfficial`, percentual de falantes – `Percentage`).

2 Consultas básicas e com WHERE

Exemplo

Países da América do Sul:

```
SELECT Name, Continent, Population
FROM country
WHERE Continent = 'South America';
```

Cidades brasileiras com população acima de 2 milhões de habitantes:

```
SELECT Name, District, Population
FROM city
WHERE CountryCode = 'BRA' AND Population > 2000000
ORDER BY Population DESC;
```

3 Funções de agregação simples

Exemplo

População total do mundo, segundo a tabela `country`:

```
SELECT SUM(Population) AS populacao_mundial
FROM country;
```

Maior e menor área entre os países da Europa:

```
SELECT MAX(SurfaceArea) AS maior_area, MIN(SurfaceArea) AS menor_area
FROM country
WHERE Continent = 'Europe';
```

4 GROUP BY e HAVING

Exemplo

População total e número de países por continente:

```
SELECT Continent,
       COUNT(*) AS qtd_paises,
       SUM(Population) AS populacao_total
FROM country
GROUP BY Continent
ORDER BY populacao_total DESC;
```

Continentes cuja expectativa de vida média ultrapassa 75 anos:

```
SELECT Continent, ROUND(AVG(LifeExpectancy), 1) AS vida_media
FROM country
GROUP BY Continent
HAVING AVG(LifeExpectancy) > 75;
```

5 Junções entre country e city

Exemplo

Nome da cidade, do país e a população de cada capital do mundo (junção `country.Capital = city.ID`):

```
SELECT co.Name AS pais, ci.Name AS capital, ci.Population AS populacao
FROM country co
JOIN city ci ON co.Capital = ci.ID
ORDER BY ci.Population DESC
LIMIT 10;
```

6 Junção com countrylanguage

Exemplo

Idiomas oficiais falados no Brasil:

```
SELECT co.Name AS pais, cl.Language, cl.Percentage
FROM country co
JOIN countrylanguage cl ON co.Code = cl.CountryCode
WHERE co.Name = 'Brazil' AND cl.IsOfficial = 'T';
```

Países em que o português é falado, com o respectivo percentual de falantes:

```
SELECT co.Name AS pais, cl.Percentage AS percentual_falantes
FROM countrylanguage cl
JOIN country co ON cl.CountryCode = co.Code
WHERE cl.Language = 'Portuguese'
ORDER BY cl.Percentage DESC;
```

7 Subconsultas no banco world

Exemplo

Países com população maior que a população média de todos os países (subconsulta não-relacionada):

```
SELECT Name, Population
FROM country
WHERE Population > (SELECT AVG(Population) FROM country)
ORDER BY Population DESC;
```

Para cada continente, o país mais populoso (subconsulta correlacionada):

```
SELECT c1.Continent, c1.Name, c1.Population
FROM country c1
WHERE c1.Population = (
    SELECT MAX(c2.Population)
    FROM country c2
    WHERE c2.Continent = c1.Continent
)
ORDER BY c1.Population DESC;
```

Países que não possuem nenhuma cidade cadastrada com mais de 1 milhão de habitantes (`NOT EXISTS`):

```
SELECT co.Name
FROM country co
WHERE NOT EXISTS (
    SELECT 1
    FROM city ci
    WHERE ci.CountryCode = co.Code
      AND ci.Population > 1000000
);
```

8 VIEW no banco world

Exemplo

```sql
CREATE VIEW populacao_por_continente AS
SELECT Continent,
       SUM(Population) AS populacao_total,
       COUNT(*) AS qtd_paises
FROM country
GROUP BY Continent;

-- Continentes com mais de 1 bilhao de habitantes
SELECT *
FROM populacao_por_continente
WHERE populacao_total > 1000000000
ORDER BY populacao_total DESC;
```

9 Banco de Dados Utilizado neste Roteiro

Banco de Dados Utilizado

Este roteiro usa o banco *world*. O script completo, com dados reais e completos (239 países, 4079 cidades e 984 registros de idiomas), está no arquivo **world.sql**, fornecido junto com este roteiro.

```
mysql -u seu_usuario -p < world.sql
```

10 Exercícios

Exercício

1. Liste os 10 países mais populosos do mundo.
2. Para cada país, mostre quantas cidades estão cadastradas na tabela `city` (use `JOIN` com `GROUP BY`).
3. Liste os países em que o espanhol (`Spanish`) é idioma oficial, ordenados por população.
4. Usando uma CTE, calcule a população total por região (`Region`) e depois liste apenas as regiões com mais de 5 países.
5. Escreva uma consulta que mostre, para cada país, o nome da capital e quantos idiomas oficiais são falados nele.

11 Solução

Solução

1. 10 países mais populosos:

```
SELECT Name, Population
FROM country
ORDER BY Population DESC
LIMIT 10;
```

2. Quantidade de cidades cadastradas por país:

```
SELECT co.Name AS pais, COUNT(ci.ID) AS qtd_cidades
FROM country co
JOIN city ci ON ci.CountryCode = co.Code
GROUP BY co.Name
ORDER BY qtd_cidades DESC;
```

3. Países em que o espanhol é idioma oficial, por população:

```
SELECT co.Name AS pais, co.Population
FROM country co
JOIN countrylanguage cl ON co.Code = cl.CountryCode
WHERE cl.Language = 'Spanish' AND cl.IsOfficial = 'T'
ORDER BY co.Population DESC;
```

4. População total por região (CTE), regiões com mais de 5 países:

```
WITH pop_regiao AS (
    SELECT Region,
           SUM(Population) AS populacao_total,
           COUNT(*) AS qtd_paises
    FROM country
    GROUP BY Region
)
SELECT *
FROM pop_regiao
WHERE qtd_paises > 5
ORDER BY populacao_total DESC;
```

5. Capital e quantidade de idiomas oficiais por país:

```
SELECT co.Name AS pais, ci.Name AS capital,
       COUNT(cl.Language) AS qtd_idiomas_oficiais
FROM country co
JOIN city ci ON co.Capital = ci.ID
LEFT JOIN countrylanguage cl
       ON cl.CountryCode = co.Code AND cl.IsOfficial = 'T'
GROUP BY co.Name, ci.Name
ORDER BY qtd_idiomas_oficiais DESC;
```

Compilado em 8 de agosto de 2026 – 10:20:02.

Revisão 1: Consultas Básicas no Mysql

fantini@coltec.ufmg

21 de agosto de 2026

Sumário

1 Apresentação **2**

2 Revisão Consultas Básicas **3**

2.1 Exercício de Fixação 6

2.2 Tabela Produtos 8

3 Revisitando Operadores Lógicos **10**

3.1 Sintaxe dos Operadores AND, OR, NOT 10

4 Exercícios de Fixação **11**

5 Valores NULL **13**

5.1 Como testar valores NULL 13

6 Funções MAX() e MIN() **15**

7 Funções COUNT(), AVG(), SUM() **16**

8 Group By **16**

8.1 Exemplos com GROUP BY 17

8.2 Group By sem Funções Agregadoras 19

8.3 Exercícios de Fixação 20

9 Soluções **21**

9.1 Exercício de Fixação 26

10 Usando a Cláusula *Having* **26**

1 Apresentação

Nesse texto apresentamos uma revisão dos conceitos básicos de consultas em uma única tabela no Mysql.

São resumidos os conceitos apresentados na nossa apostila online.

Apresentamos exemplos de consultas realizadas e exercícios, com solções. Não olhe as soluções antes de fazer as consultas pedidas.

São 32 consultas com exemplos do USO de SELECT com as diversas opções, incluindo funções de agregação, operadores e funções.

São 5 listas de "Exercícios de Fixação", todos com soluções.

Usamos tabelas criadas pelo usuário e tabelas do banco de dados *northwind2024*.

2 Revisão Consultas Básicas

Select com Distinct

A instrução SELECT DISTINCT é usada para retornar apenas valores distintos (diferentes).

A tabela 1 apresenta dados de clientes registrados em uma empresa. Veja as duas consultas que informam os estados registrados na tabela.

Tabela 1: Tabela com Informações dos Clientes

ID	Nome	Endereco	Cidade	CEP	Estado
1	Jose Maria	Rua 1, num 2	Betim	12209	MG
2	Maria Jose	Rua 2, num 33	Maceio	5021	AL
3	Ana Maria	Rua A, num 4	Aracaju	5023	SE
4	Joana Carla	Rua 6, 7	BH	123	MG
5	João Carlos	Av. Flor	BH	12345	MG

```
mysql> select distinct Estado from Clientes;
+--------+
| Estado |
+--------+
| MG     |
| AL     |
| SE     |
+--------+
```

```
mysql> select Estado from Clientes;
+--------+
| Estado |
+--------+
| MG     |
| AL     |
| SE     |
| MG     |
| MG     |
+--------+
```

Usando o COUNT com Distinct

Podemos usar a palavra-chave DISTINCT para contar quantos estados temos registrados na tabela. Veja a diferença da consulta com e sem DISTINCT.

```
mysql> select count(distinct Estado) from Clientes;
+------------------------+
| count(distinct Estado) |
+------------------------+
|                      3 |
```

```
+----------------------+

mysql> select count(Estado) from Clientes;
+---------------+
| count(Estado) |
+---------------+
|             5 |
+---------------+
```

SELECT com WHERE

A cláusula WHERE é usada para filtrar registros.

É usado para extrair apenas os registros que atendem a uma condição especificada.

Selecione os clientes do Estado de Minas Gerais. Mostre todos os campos.

```
mysql> select * from Clientes where Estado = 'MG';
+------+--------------+-------------+--------+-------+--------+
| ID   | Nome         | Endereco    | Cidade | CEP   | Estado |
+------+--------------+-------------+--------+-------+--------+
|    1 | Jose Maria   | Rua 1, num 2 | Betim  | 12209 | MG     |
|    4 | Joana Carla  | Rua 6, 7     | BH     |   123 | MG     |
|    5 | João Carlos  | Av. Flor     | BH     | 12345 | MG     |
+------+--------------+-------------+--------+-------+--------+
```

Podemos usar a palavra-chave **LIKE**.

```
mysql> select * from Clientes where Estado like 'MG';
+------+--------------+-------------+--------+-------+--------+
| ID   | Nome         | Endereco    | Cidade | CEP   | Estado |
+------+--------------+-------------+--------+-------+--------+
|    1 | Jose Maria   | Rua 1, num 2 | Betim  | 12209 | MG     |
|    4 | Joana Carla  | Rua 6, 7     | BH     |   123 | MG     |
|    5 | João Carlos  | Av. Flor     | BH     | 12345 | MG     |
+------+--------------+-------------+--------+-------+--------+
3 rows in set (0.00 sec)
```

Obviamente podemos fazer consultas com condições para números também.

```
mysql> select * from Clientes where ID = 1;
+------+------------+-------------+--------+-------+--------+
| ID   | Nome       | Endereco    | Cidade | CEP   | Estado |
+------+------------+-------------+--------+-------+--------+
|    1 | Jose Maria | Rua 1, num 2 | Betim  | 12209 | MG     |
+------+------------+-------------+--------+-------+--------+
1 row in set (0.00 sec)
```

- Consultas com strings, usamos aspas duplas ou simples.
- Consultas com números, não usamos aspas.

Consultas com WHERE + operadores

Podemos usar os operadores de comparação:

- igual, =;
- maior, maior ou igual: >, >=;
- diferente, ! = ou <>;
- menor, menor ou igual: <, <=;

Podemos também usar as palavras chaves:

- BETWEEN
- LIKE
- IN

Podemos também usar funções matemáticas para filtrar dados. Uma função muito usada é a MOD() que retorna o resto da divisão inteira. Usamos MOD para, por exemplo, verificar se um número é par ou ímpar.

```
mysql> select mod(4,2);
+----------+
| mod(4,2) |
+----------+
|        0 |
+----------+
mysql> select mod(5,2);
+----------+
| mod(5,2) |
+----------+
|        1 |
+----------+
```

2.1 Exercício de Fixação

Faças as Consultas Pedidas

1. Selecione todos clientes com ID < 3. Mostre ID e Nome.
2. Selecione todos clientes com ID par. Mostre ID e Nome.
3. Selecione todos clientes cuja cidade começa com letra "B". Mostre ID, Nome e Cidade.
4. Selecione todos clientes com ID entre 3 e 5. Mostre ID e Nome.
5. Selecione todos os clientes dos estados do Nordeste. Mostre ID,Nome e Estado.
6. Selecione Nome, ID e Cidade dos clientes que se chamam João ou Joana.

Sugestão de Solução

```
mysql> #1) selecione todos clientes com ID < 3
mysql> select ID, Nome from Clientes where ID < 3;
+------+------------+
| ID   | Nome       |
+------+------------+
|    1 | Jose Maria |
|    2 | Maria Jose |
+------+------------+
2 rows in set (0.00 sec)

mysql> # 2) Selecione todos clientes com ID par. Mostre ID e Nome.
mysql> select ID,Nome from Clientes
    -> where mod(ID,2) = 0;
+------+-------------+
| ID   | Nome        |
+------+-------------+
|    2 | Maria Jose  |
|    4 | Joana Carla |
+------+-------------+
2 rows in set (0.00 sec)


mysql> # 3) Selecione todos clientes cuja cidade começa com letra "B". Mostre ID, Nome e C
mysql> select ID,Nome,Cidade from Clientes  where Cidade LIKE "B%";
+------+--------------+--------+
| ID   | Nome         | Cidade |
+------+--------------+--------+
|    1 | Jose Maria   | Betim  |
|    4 | Joana Carla  | BH     |
|    5 | João Carlos  | BH     |
+------+--------------+--------+
3 rows in set (0.00 sec)
```

```
mysql> # 4) Selecione todos clientes com ID entre 3 e 5. Mostre ID e Nome.
mysql> select ID,Nome,Cidade from Clientes  where ID BETWEEN 3 AND 5;
+------+--------------+---------+
| ID   | Nome         | Cidade  |
+------+--------------+---------+
|    3 | Ana Maria    | Aracaju |
|    4 | Joana Carla  | BH      |
|    5 | João Carlos  | BH      |
+------+--------------+---------+
3 rows in set (0.00 sec)
```

```
mysql> # 5) Selecione todos os clientes dos estados do Nordeste. Mostre ID,Nome e Estado.
mysql> select ID,Nome,Estado from Clientes  where Estado IN ('AL','SE');
+------+------------+--------+
| ID   | Nome       | Estado |
+------+------------+--------+
|    2 | Maria Jose | AL     |
|    3 | Ana Maria  | SE     |
+------+------------+--------+
2 rows in set (0.00 sec)
```

```
mysql> # 6) Selecione Nome, ID e Cidade dos clientes que se chamam João ou Joana.
mysql> select Nome,ID, Cidade from Clientes  where Nome LIKE "João%" or Nome LIKE "Joana%"
+--------------+------+--------+
| Nome         | ID   | Cidade |
+--------------+------+--------+
| Joana Carla  |    4 | BH     |
| João Carlos  |    5 | BH     |
+--------------+------+--------+
2 rows in set (0.00 sec)
```

- Repare que na questão 5 usamos a palavra-chave IN para verificar se o estado pesquisado está dentro do conjunto definido como ('AL','SE'). Essa é uma forma comum de avaliarmos se um valor está dentro de um conjunto de valores.
- Podemos usar os operadores lógicos AND, OR para pesquisarmos valores.

```
mysql> # 7) Selecione Nome, ID e Cidade dos clientes que se chamam João ou Joana.
mysql> select Nome,ID,Cidade from Clientes where Nome like "João%" or Nome Like "Joana%";
+--------------+------+--------+
| Nome         | ID   | Cidade |
+--------------+------+--------+
| Joana Carla  |    4 | BH     |
| João Carlos  |    5 | BH     |
+--------------+------+--------+
2 rows in set (0.00 sec)
```

A consulta a seguir usa o operador AND:

7. Mostre os clientes que tem nome que começam com letra "J"e moram em cidades que tem nome que começam com letra "B". Mostre o nome e a cidade.

```
mysql> # 8)  Mostre os clientes que tem nome que começam com letra "J"
e moram em cidades que tem nome que começam com letra "B".
Mostre o nome e a cidade.
mysql> select Nome,Cidade from Clientes where Nome like "J%" and Cidade Like "B%";
+--------------+--------+
| Nome         | Cidade |
+--------------+--------+
| Jose Maria   | Betim  |
| Joana Carla  | BH     |
| João Carlos  | BH     |
+--------------+--------+
3 rows in set (0.00 sec)
```

2.2 Tabela Produtos

Tabela 2: Tabela com Produtos

ID	ProductName	CategoriaID	Quantidade	Preco
1	HD	1	12	400
2	PlacaMae	1	15	1000
3	Memória RAM	2	14	500
4	Papel A4	3	33	40
5	Canetas	4	44	2
6	Marcador	4	22	3

Palavra-chave ORDER BY

Usamos ORDER BY para order em ordem crescente (ASC) ou decrescente (DESC).

8. Informe todos os produtos, ordenados pelo preço, em ordem descrescente.

```
mysql> # 8) Informe todos os produtos, ordenados pelo preço, em ordem descrescente.
mysql> select * from Produtos order by Preco desc;
+------+-------------+------------+-----------+-------+
| ID   | ProductName | CategoriaID | Quantidade | Preco |
+------+-------------+------------+-----------+-------+
|    2 | PlacaMae    |          1 |         15 |  1000 |
|    3 | Memória RAM |          2 |         14 |   500 |
|    1 | HD          |          1 |         12 |   400 |
|    4 | Papel A4    |          3 |         33 |    40 |
|    6 | Marcador    |          4 |         22 |     3 |
|    5 | Canetas     |          4 |         44 |     2 |
+------+-------------+------------+-----------+-------+
6 rows in set (0,00 sec)
```

9. Informe todos os produtos, ordenados pelo preço, em ordem crescente.

```
mysql> select * from Produtos order by Preco asc;
+------+-------------+------------+-----------+-------+
| ID   | ProductName | CategoriaID | Quantidade | Preco |
+------+-------------+------------+-----------+-------+
|    5 | Canetas     |          4 |         44 |     2 |
|    6 | Marcador    |          4 |         22 |     3 |
|    4 | Papel A4    |          3 |         33 |    40 |
|    1 | HD          |          1 |         12 |   400 |
|    3 | Memória RAM |          2 |         14 |   500 |
|    2 | PlacaMae    |          1 |         15 |  1000 |
+------+-------------+------------+-----------+-------+
6 rows in set (0,00 sec)
```

Podemos ordenar por mais de uma coluna, como mostra a próxima consulta.

10. Mostre todos os clientes, ordenados por estados e por nome.

```
mysql> # 10) Mostre todos os clientes, ordenados por estados e por nome.
mysql> select * from Clientes order by Estado, Nome;
+------+-------------+--------------+---------+-------+--------+
| ID   | Nome        | Endereco     | Cidade  | CEP   | Estado |
+------+-------------+--------------+---------+-------+--------+
|    2 | Maria Jose  | Rua 2, num 33 | Maceio  |  5021 | AL     |
|    4 | Joana Carla | Rua 6, 7     | BH      |   123 | MG     |
|    5 | João Carlos | Av. Flor     | BH      | 12345 | MG     |
|    1 | Jose Maria  | Rua 1, num 2 | Betim   | 12209 | MG     |
|    3 | Ana Maria   | Rua A, num 4 | Aracaju |  5023 | SE     |
+------+-------------+--------------+---------+-------+--------+
```

3 Revisitando Operadores Lógicos

Podemos usar combinações de Operadores Lógicos: AND, OR, NOT. Acompanhe as consultas a seguir.

11. Encontre os clientes de Minas Gerais que possuem o último nome "Maria"ou "Carla".

```
mysql> # 11) Encontre os clientes de Minas Gerais que possuem
o último nome "Maria" ou "Carla".
mysql> select * from Clientes
-> where Estado = 'MG' and (Nome like "%Maria" or Nome like "%Carla");
+------+-------------+--------------+--------+-------+--------+
| ID   | Nome        | Endereco     | Cidade | CEP   | Estado |
+------+-------------+--------------+--------+-------+--------+
|    1 | Jose Maria  | Rua 1, num 2 | Betim  | 12209 | MG     |
|    4 | Joana Carla | Rua 6, 7     | BH     |   123 | MG     |
+------+-------------+--------------+--------+-------+--------+
```

12. Encontre os Clientes que não são de Minas Gerais

```
mysql> # 12) Encontre os Clientes que não são de Minas Gerais
mysql> select * from Clientes where NOT Estado = 'MG';
+------+------------+---------------+---------+------+--------+
| ID   | Nome       | Endereco      | Cidade  | CEP  | Estado |
+------+------------+---------------+---------+------+--------+
|    2 | Maria Jose | Rua 2, num 33 | Maceio  | 5021 | AL     |
|    3 | Ana Maria  | Rua A, num 4  | Aracaju | 5023 | SE     |
+------+------------+---------------+---------+------+--------+
```

3.1 Sintaxe dos Operadores AND, OR, NOT

AND

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...;
```

OR

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 OR condition2 OR condition3 ...;
```

NOT

```
SELECT column1, column2, ...
FROM table_name
WHERE NOT condition;
```

4 Exercícios de Fixação

(adaptado de W3Schools: `www.w3schools.com/sql/sql_and.asp`)

Usando o Banco de Dados northwind2024, faça as consultas pedidas:

1. Mostre os clientes (Customers) que são da Alemanha e da cidade Berlin. Mostre os campos CustomerID,CompanyName,City,Country.

```
use northwind2024
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> WHERE Country = 'Germany' AND City = 'Berlin';
+------------+---------------------+--------+---------+
| CustomerID | CompanyName         | City   | Country |
+------------+---------------------+--------+---------+
| ALFKI      | Alfreds Futterkiste | Berlin | Germany |
+------------+---------------------+--------+---------+
```

2. Mostre os clientes que são de Berlin ou de Stuttgart.

```
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> where City = "Berlin" OR City = "Stuttgart";
+------------+---------------------+-----------+---------+
| CustomerID | CompanyName         | City      | Country |
+------------+---------------------+-----------+---------+
| ALFKI      | Alfreds Futterkiste | Berlin    | Germany |
| WANDK      | Die Wandernde Kuh   | Stuttgart | Germany |
+------------+---------------------+-----------+---------+
```

3. Mostre os clientes que são da Espanha ou da Alemanha.

```
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> WHERE Country = "Germany" OR Country = "Spain";
+------------+-------------------------------------+----------------+---------+
| CustomerID | CompanyName                         | City           | Country |
+------------+-------------------------------------+----------------+---------+
| ALFKI      | Alfreds Futterkiste                 | Berlin         | Germany |
| BLAUS      | Blauer See Delikatessen             | Mannheim       | Germany |
| BOLID      | Blido Comidas preparadas            | Madrid         | Spain   |
:
:
:
| ROMEY      | Romero y tomillo                    | Madrid         | Spain   |
| TOMSP      | Toms Spezialitten                   | Mnster         | Germany |
| WANDK      | Die Wandernde Kuh                   | Stuttgart      | Germany |
+------------+-------------------------------------+----------------+---------+
16 rows in set (0,00 sec)
```

4. Selecione os clientes que não são da Alemanha.

```
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> where NOT Country = "Germany";
+------------+--------------------------------------+-----------------+-------------+
| CustomerID | CompanyName                          | City            | Country     |
+------------+--------------------------------------+-----------------+-------------+
| ANATR      | Ana Trujillo Emparedados y helados   | Mxico D.F.      | Mexico      |
:
:
:
| WELLI      | Wellington Importadora               | Resende         | Brazil      |
| WHITC      | White Clover Markets                 | Seattle         | USA         |
| WILMK      | Wilman Kala                          | Helsinki        | Finland     |
| WOLZA      | Wolski  Zajazd                       | Warszawa        | Poland      |
+------------+--------------------------------------+-----------------+-------------+
80 rows in set (0,00 sec)
```

5. Mostre os clientes que não são da Alemanha nem dos Estados Unidos.

```
mysql> SELECT CustomerID,CompanyName,City,Country FROM Customers
    -> WHERE NOT Country = 'Germany' AND NOT Country = 'USA';
+------------+--------------------------------------+-----------------+-------------+
| CustomerID | CompanyName                          | City            | Country     |
+------------+--------------------------------------+-----------------+-------------+
| ANATR      | Ana Trujillo Emparedados y helados   | Mxico D.F.      | Mexico      |
| ANTON      | Antonio Moreno Taquera               | Mxico D.F.      | Mexico      |
:
:
| WELLI      | Wellington Importadora               | Resende         | Brazil      |
| WILMK      | Wilman Kala                          | Helsinki        | Finland     |
| WOLZA      | Wolski  Zajazd                       | Warszawa        | Poland      |
+------------+--------------------------------------+-----------------+-------------+
67 rows in set (0,00 sec)
```

5 Valores NULL

(adaptado de W3Schools: `https://www.w3schools.com/mysql/mysql_null_values.asp`)

Um campo com valor NULL é um campo sem valor.

Se um campo de uma tabela for opcional, é possível inserir um novo registro ou atualizar um registro sem adicionar um valor a este campo. Então, o campo será salvo com valor NULL.

Atenção: Um valor NULL é diferente de um valor zero ou de um campo que contém espaços. Um campo com valor NULL é aquele que foi deixado em branco durante a criação do registro!

5.1 Como testar valores NULL

Para testar se um valoe é ou não NULL, não podemos usar os operadores de comparação, como =, < ou <>.

Para testar se um valor é NULL temos que usar os operadores **IS NULL** e **IS NOT NULL**.

SINTAXE

```
SELECT column_names
FROM table_name
WHERE column_name IS NULL;
```

ou, para NOT NULL:

```
SELECT column_names
FROM table_name
WHERE column_name IS NOT NULL;
```

Acompanhe as consultas para o banco northwind2024.

13. Mostre o ID, Nome e Endereço dos Clientes que possuem NULL para o campo "Address".

```
mysql> # 13) Mostre o Nome e Endereço dos Clientes que possuem NULL para o campo ''Ad
+------------+-------------+---------+
| CustomerID | ContactName | Address |
+------------+-------------+---------+
| Val2       | Val2        | NULL    |
| VALON      | Valon Hoti  | NULL    |
+------------+-------------+---------+
2 rows in set (0,00 sec)
```

14. Mostre o ID, Nome e Endereço dos Clientes que possuem valores para o campo "Address".

```
# 14) Mostre o ID, Nome e Endereço dos Clientes que possuem valores para o campo ''Ad
mysql> select CustomerID, ContactName, Address
    -> from Customers
    -> where Address IS NOT NULL;
+------------+------------------------+-------------------------------------------
| CustomerID | ContactName            | Address
+------------+------------------------+-------------------------------------------
| ALFKI      | Maria Anders           | Obere Str. 57
| ANATR      | Ana Trujillo           | Avda. de la Constitucin 2222
| ANTON      | Antonio Moreno         | Mataderos  2312
:
:
| WHITC      | Karl Jablonski         | 305 - 14th Ave. S. Suite 3B
| WILMK      | Matti Karttunen        | Keskuskatu 45
| WOLZA      | Zbyszek Piestrzeniewicz | ul. Filtrowa 68
+------------+------------------------+-------------------------------------------
91 rows in set (0,00 sec)
```

6 Funções MAX() e MIN()

Vamos continuar usando o banco de dados northwind2024.

Veja os 15 primeiros itens da tabela "Products", mostrando os campos ProductID, ProductName, UnitsInStock,UnitPrice.

```
mysql> select ProductID, ProductName, UnitsInStock,UnitPrice from Products limit 15;
+-----------+---------------------------------+--------------+-----------+
| ProductID | ProductName                     | UnitsInStock | UnitPrice |
+-----------+---------------------------------+--------------+-----------+
|         1 | Chai                            |           39 |   18.0000 |
|         2 | Chang                           |           17 |   19.0000 |
|         3 | Aniseed Syrup                   |           13 |   10.0000 |
|         4 | Chef Anton's Cajun Seasoning    |           53 |   22.0000 |
|         5 | Chef Anton's Gumbo Mix          |            0 |   21.3500 |
|         6 | Grandma's Boysenberry Spread    |          120 |   25.0000 |
|         7 | Uncle Bob's Organic Dried Pears |           15 |   30.0000 |
|         8 | Northwoods Cranberry Sauce      |            6 |   40.0000 |
|         9 | Mishi Kobe Niku                 |           29 |   97.0000 |
|        10 | Ikura                           |           31 |   31.0000 |
|        11 | Queso Cabrales                  |           22 |   21.0000 |
|        12 | Queso Manchego La Pastora       |           86 |   38.0000 |
|        13 | Konbu                           |           24 |    6.0000 |
|        14 | Tofu                            |           35 |   23.2500 |
|        15 | Genen Shouyu                    |           39 |   15.5000 |
+-----------+---------------------------------+--------------+-----------+
15 rows in set (0,00 sec)
```

15. Mostre o menor preço dentre todos os preços de todos produtos.

```
mysql> SELECT MIN(UnitPrice) AS "Menor Preço" FROM Products;
+--------------+
| Menor Preço  |
+--------------+
|       2.5000 |
+--------------+
1 row in set (0,00 sec)
```

16. Mostre o preço mais alto dentre todos os preços de todos produtos.

```
mysql> SELECT MAX(UnitPrice) AS "Maior Preço" FROM Products;
+--------------+
| Maior Preço  |
+--------------+
|     263.5000 |
+--------------+
1 row in set (0,01 sec)
```

7 Funções COUNT(), AVG(), SUM()

Seguem exemplos dessas três funções aplicadas a consultas no banco de dados northwind2024.

A função COUNT() retorna o número de linhas correspondentes a um critério especificado.

17. Informe quantos produtos há na tabela “Products”;

```
mysql> # 17) Informe quantos produtos há na tabela ‘‘Products’’;
mysql> SELECT COUNT(ProductID)
    -> FROM Products;
+------------------+
| COUNT(ProductID) |
+------------------+
|               77 |
+------------------+
1 row in set (0.00 sec)
```

18. Informe o valor médio dentre todos os preços informados na tabela “Products”;

```
mysql> # 18) Informe o valor médio dentre todos os preços informados na tabela ‘‘Prod
mysql> SELECT AVG(UnitPrice) FROM Products;
+-----------------+
| AVG(UnitPrice) |
+-----------------+
|    28.86636364 |
+-----------------+
1 row in set (0.00 sec)
```

19. Encontre total de itens que foram pedidos. Use a tabela OrderDetails, verificando a coluna ”Quantity”.

```
mysql> # 19) Encontre total de itens que foram pedidos. Use a tabela OrderDetails, ve
mysql> SELECT SUM(Quantity) FROM OrderDetails;
+---------------+
| SUM(Quantity) |
+---------------+
|         51317 |
+---------------+
1 row in set (0.00 sec)
```

8 Group By

Usamos a cláusula GROUP BY para agrupar registros em subgrupos baseados em colunas ou valores retornados por uma expressão.

Com o GROUP BY podemos agrupar os valores de uma coluna e também realizar cálculos sobre esses valores. Desta forma, ao realizarmos ma consulta, os valores encontrados nas linhas são agrupados e então uma função de agregação pode ser aplicada sobre esses grupos.

A instrução GROUP BY é frequentemente usada com funções agregadoras (COUNT(), MAX(), MIN(), SUM(), AVG()) para agrupar o conjunto de resultados por uma ou mais colunas.

A sintaxe básica para uso do **group by** é:

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

8.1 Exemplos com GROUP BY

20. Encontre o número total de clientes de cada país.

```
mysql> SELECT COUNT(CustomerID), Country
    -> FROM Customers
    -> GROUP BY Country;
+-------------------+-------------+
| COUNT(CustomerID) | Country     |
+-------------------+-------------+
|                11 | Germany     |
|                 5 | Mexico      |
|                 7 | UK          |
:
:
|                 2 | NULL        |
|                 2 | Finland     |
|                 1 | Poland      |
+-------------------+-------------+
22 rows in set (0,00 sec)
```

21. Podemos ordenar a consulta acima:

```
mysql> SELECT COUNT(CustomerID), Country FROM Customers GROUP BY Country
    -> order by Country ASC;
+-------------------+-------------+
| COUNT(CustomerID) | Country     |
+-------------------+-------------+
|                 2 | NULL        |
|                 3 | Argentina   |
|                 2 | Austria     |
|                 2 | Belgium     |
:
:
|                 7 | UK          |
|                13 | USA         |
|                 4 | Venezuela   |
+-------------------+-------------+
22 rows in set (0,00 sec)
```

22. Podemos ordenar em relação ao número de clientes:

```
mysql> SELECT COUNT(CustomerID) as "Número Clientes", Country FROM Customers GROUP BY
+------------------+-------------+
| Número Clientes  | Country     |
+------------------+-------------+
|                1 | Poland      |
|                1 | Norway      |
|                1 | Ireland     |
|                2 | Finland     |
|                2 | Sweden      |
|                2 | NULL        |
|                2 | Denmark     |
|                2 | Belgium     |
|                2 | Switzerland |
|                2 | Austria     |
|                2 | Portugal    |
|                3 | Italy       |
|                3 | Argentina   |
|                3 | Canada      |
|                4 | Venezuela   |
|                5 | Spain       |
|                5 | Mexico      |
|                7 | UK          |
|                9 | Brazil      |
|               11 | Germany     |
|               11 | France      |
|               13 | USA         |
+------------------+-------------+
22 rows in set (0,00 sec)
```

8.2 Group By sem Funções Agregadoras

Não é comum usarmos o group by sem ter uma agregação (como o cálculo da soma, da média, ou a busca de valores máximos ou mínimos). Geralmente agrupamos algo quando queremos ver valores totais ou médias do grupo, por exemplo.

Uma situação de consulta com group by sem função de agregação é para buscarmos valores distintos. Veja a consulta a seguir, realizada no banco de dados northwind2024.

```
mysql> select FirstName, LastName, Address, City, Country from Employees;
+-----------+-----------+--------------------------------+----------+---------+
| FirstName | LastName  | Address                        | City     | Country |
+-----------+-----------+--------------------------------+----------+---------+
| Nancy     | Davolio   | 507 - 20th Ave. E.Apt. 2A      | Seattle  | USA     |
| Andrew    | Fuller    | 908 W. Capital Way             | Tacoma   | USA     |
| Janet     | Leverling | 722 Moss Bay Blvd.             | Kirkland | USA     |
| Margaret  | Peacock   | 4110 Old Redmond Rd.           | Redmond  | USA     |
| Steven    | Buchanan  | 14 Garrett Hill                | London   | UK      |
| Michael   | Suyama    | Coventry House
Miner Rd.       | London   | UK      |
| Robert    | King      | Edgeham Hollow
Winchester Way | London   | UK      |
| Laura     | Callahan  | 4726 - 11th Ave. N.E.          | Seattle  | USA     |
| Anne      | Dodsworth | 7 Houndstooth Rd.              | London   | UK      |
+-----------+-----------+--------------------------------+----------+---------+
9 rows in set (0,00 sec)
```

Se quisermos agrupar por país nossa consulta podemos fazer:

```
mysql> select Country from Employees group by Country;
+---------+
| Country |
+---------+
| USA     |
| UK      |
+---------+
2 rows in set (0,00 sec)
```

O que é a mesma coisa de fazermos:

```
mysql> select DISTINCT Country from Employees;
+---------+
| Country |
+---------+
| USA     |
| UK      |
+---------+
2 rows in set (0,00 sec)
```

Agora, se tentarmos agrupar por país e mostrarmos por exemplo o primeiro nome de cada empregado, geraríamos um erro:

```
mysql> select FirstName,Country from Employees group by Country;
ERROR 1055 (42000): Expression #1 of SELECT list is not in GROUP BY clause
and contains nonaggregated column 'northwind2024.Employees.FirstName'
which is not functionally dependent on columns in GROUP BY clause;
this is incompatible with sql_mode=only_full_group_by
```

8.3 Exercícios de Fixação

1. Mostre os IDs das regiões e o total de territórios de cada uma.
2. Mostre as diferentes categorias de produtos e o valor total dos preços de cada uma.
3. Mostre quantos registros possui a tabela OrderDetails;
4. Mostre quantos pedidos foram feitos, verificando quantos pedidos estão registrados na tabela OrderDetails. Use o alias "Total de Pedidos" para o resultado.
5. Mostre quantos pedidos foram feitos, consultando a tabela Orders.
6. Liste os pedidos feitos, informando o ID do pedido (OrderID), o preço de cada unidade e a quantidade pedida. Use a tabela OrderDetails.
7. Repita a consulta acima, limitando a consulta aos 10 primeiros registros.
8. Repita a consulta dada no item 6, limitando aos 10 últimos registros.
9. Informe o número do pedido, o valor unitário, quantidade e o preço total de cada pedido. O preço total é dado por (preço unitário) $\times$ (quantidade).
10. Refaça a consulta anterior, agora mostrando os valores totais de Preço Unitário, de Quantidade e do Preço Total para cada OrderID (ou seja agrupado por número de pedido).
11. Refaça a consulta anterior, agora mostrando o 10 maiores "Preços Totais".
12. Faça uma consulta mostrando o ID do pedido (OrderID) e o preço total do pedido. Use o apelido "Preço Total". Mostre os 10 primeiros pedidos com maiores valores totais

9 Soluções

Sugestões de soluções para os exercícios da seção 8.3.

```
# Exercício de Fixação
# 1) Mostre os IDs das regiões e o total de territórios de cada uma.
select RegionID,count(TerritoryID) from Territories group by RegionID;
+----------+--------------------+
| RegionID | count(TerritoryID) |
+----------+--------------------+
|        1 |                 19 |
|        2 |                 15 |
|        3 |                 11 |
|        4 |                  8 |
+----------+--------------------+
4 rows in set (0,00 sec)

# 2) Mostre as diferentes categorias de produtos e o valor total dos preços de cada uma.
select CategoryID,sum(UnitPrice) from Products group by CategoryID;
+------------+----------------+
| CategoryID | sum(UnitPrice) |
+------------+----------------+
|          1 |       455.7500 |
|          2 |       276.7500 |
|          3 |       327.0800 |
|          4 |       287.3000 |
|          5 |       141.7500 |
|          6 |       324.0400 |
|          7 |       161.8500 |
|          8 |       248.1900 |
+------------+----------------+
8 rows in set (0,00 sec)

#  3) Mostre quantos registros possui a tabela OrderDetails;
mysql> select count(*) from OrderDetails;
+----------+
| count(*) |
+----------+
|     2155 |
+----------+
1 row in set (0,00 sec)

# 4. Mostre quantos pedidos foram feitos, verificando quantos pedidos estão registrados na
OrderDetails. Use o alias ''Total de Pedidos'' para o resultado.

mysql> select count(distinct OrderID) as "Total de Pedidos" from OrderDetails;
+------------------+
| Total de Pedidos |
+------------------+
|              830 |
+------------------+
1 row in set (0,00 sec)
```

```
# 5) Mostre quantos pedidos foram feitos, consultando a tabela Orders.
mysql> select count(*) 'Total de Pedidos' from Orders;
+------------------+
| Total de Pedidos |
+------------------+
|              830 |
+------------------+
1 row in set (0,00 sec)
ou
mysql> select count(Distinct OrderID)  'Total de Pedidos'
    -> from Orders;
+------------------+
| Total de Pedidos |
+------------------+
|              830 |
+------------------+

# 6) Liste os pedidos feitos, informando o ID do pedido (OrderID),
o preço de cada unidade e a quantidade pedida. Use a tabela OrderDetails.

mysql> select OrderID, UnitPrice, Quantity  from OrderDetails;
+---------+-----------+----------+
| OrderID | UnitPrice | Quantity |
+---------+-----------+----------+
|   10248 |   14.0000 |       12 |
|   10248 |    9.8000 |       10 |
|   10248 |   34.8000 |        5 |
|   10249 |   18.6000 |        9 |
|   10249 |   42.4000 |       40 |
|   10250 |    7.7000 |       10 |
|   10250 |   42.4000 |       35 |
|   10250 |   16.8000 |       15 |
|   10251 |   16.8000 |        6 |
|   10251 |   15.6000 |       15 |
:
:
:
|   11077 |   33.2500 |        2 |
|   11077 |   17.0000 |        1 |
|   11077 |   15.0000 |        2 |
|   11077 |    7.7500 |        4 |
|   11077 |   13.0000 |        2 |
+---------+-----------+----------+
2155 rows in set (0,00 sec)
```

```
# 7)  Repita a consulta acima, limitando a consulta aos 10 primeiros registros.
mysql> select OrderID, UnitPrice, Quantity  from OrderDetails limit 10;
+---------+-----------+----------+
| OrderID | UnitPrice | Quantity |
+---------+-----------+----------+
|   10248 |   14.0000 |       12 |
|   10248 |    9.8000 |       10 |
|   10248 |   34.8000 |        5 |
|   10249 |   18.6000 |        9 |
|   10249 |   42.4000 |       40 |
|   10250 |    7.7000 |       10 |
|   10250 |   42.4000 |       35 |
|   10250 |   16.8000 |       15 |
|   10251 |   16.8000 |        6 |
|   10251 |   15.6000 |       15 |
+---------+-----------+----------+
10 rows in set (0,00 sec)


# 8) Repita a  consulta dada no item 6, limitando aos 10 últimos registros.

mysql> select OrderID, UnitPrice, Quantity  from OrderDetails limit 2145,10;
+---------+-----------+----------+
| OrderID | UnitPrice | Quantity |
+---------+-----------+----------+
|   11077 |    9.6500 |        3 |
|   11077 |   12.0000 |        3 |
|   11077 |    7.0000 |        2 |
|   11077 |   24.0000 |        2 |
|   11077 |   34.0000 |        2 |
|   11077 |   33.2500 |        2 |
|   11077 |   17.0000 |        1 |
|   11077 |   15.0000 |        2 |
|   11077 |    7.7500 |        4 |
|   11077 |   13.0000 |        2 |
+---------+-----------+----------+
10 rows in set (0,01 sec)

# repare o uso do limit. Nesse caso fizemos a consulta a partir do registro 2145, e limita
```

9) Informe o número do pedido, o valor unitário, quantidade e o preço total de cada pedido

```
select OrderID, UnitPrice, Quantity, UnitPrice*Quantity as PrecoTotal from
OrderDetails;
+---------+-----------+----------+------------+
| OrderID | UnitPrice | Quantity | PrecoTotal |
+---------+-----------+----------+------------+
|   10248 |   14.0000 |       12 |   168.0000 |
|   10248 |    9.8000 |       10 |    98.0000 |
|   10248 |   34.8000 |        5 |   174.0000 |
|   10249 |   18.6000 |        9 |   167.4000 |
|   10249 |   42.4000 |       40 |  1696.0000 |
|   10250 |    7.7000 |       10 |    77.0000 |
|   10250 |   42.4000 |       35 |  1484.0000 |
|   10250 |   16.8000 |       15 |   252.0000 |
|   10251 |   16.8000 |        6 |   100.8000 |
|   10251 |   15.6000 |       15 |   234.0000 |

:
:
:
|  11077  |   12.0000 |        3 |    36.0000 |
|   11077 |    7.0000 |        2 |    14.0000 |
|   11077 |   24.0000 |        2 |    48.0000 |
|   11077 |   34.0000 |        2 |    68.0000 |
|   11077 |   33.2500 |        2 |    66.5000 |
|   11077 |   17.0000 |        1 |    17.0000 |
|   11077 |   15.0000 |        2 |    30.0000 |
|   11077 |    7.7500 |        4 |    31.0000 |
|   11077 |   13.0000 |        2 |    26.0000 |
+---------+-----------+----------+------------+
2155 rows in set (0,01 sec)
```

```
# 10) Refaça a consulta anterior, agora mostrando os valores totais de Preço Unitário,
 de Quantidade e do Preço Total para cada  OrderID (ou seja agrupado por número de pedido)

mysql> select OrderID, Sum(UnitPrice), Sum(Quantity), Sum(UnitPrice*Quantity) as PrecoTota
 from OrderDetails group by OrderID;
+---------+----------------+---------------+------------+
| OrderID | Sum(UnitPrice) | Sum(Quantity) | PrecoTotal |
+---------+----------------+---------------+------------+
|   10248 |        58.6000 |            27 |   440.0000 |
|   10249 |        61.0000 |            49 |  1863.4000 |
|   10250 |        66.9000 |            60 |  1813.0000 |
|   10251 |        49.2000 |            41 |   670.8000 |
|   10252 |        94.0000 |           105 |  3730.0000 |
|   10253 |        40.4000 |           102 |  1444.8000 |
|   10254 |        30.8000 |            57 |   625.2000 |
|   10255 |        88.3000 |           110 |  2490.5000 |
|   10256 |        36.6000 |            27 |   517.8000 |
|   10257 |        59.9000 |            46 |  1119.9000 |
```

```
:
:
|   11071 |        36.0000 |            25 |   510.0000 |
|   11072 |        78.1500 |           200 |  5218.0000 |
|   11073 |        25.5000 |            30 |   300.0000 |
|   11074 |        17.4500 |            14 |   244.3000 |
|   11075 |        49.0000 |            42 |   586.0000 |
|   11076 |        57.4500 |            50 |  1057.0000 |
|   11077 |       574.3500 |            72 |  1374.6000 |
+---------+----------------+---------------+------------+
830 rows in set (0,00 sec)
```

11) Refaça a consulta anterior, agora mostrando o 10 maiores ‘‘Preços Totais’’.

```
mysql> select OrderID, Sum(UnitPrice), Sum(Quantity),
Sum(UnitPrice*Quantity) as PrecoTotal from OrderDetails
group by OrderID order by PrecoTotal desc limit 10;
+---------+----------------+---------------+------------+
| OrderID | Sum(UnitPrice) | Sum(Quantity) | PrecoTotal |
+---------+----------------+---------------+------------+
|   10865 |       281.5000 |           140 | 17250.0000 |
|   11030 |       219.1400 |           330 | 16321.9000 |
|   10981 |       263.5000 |            60 | 15810.0000 |
|   10372 |       330.6000 |           164 | 12281.2000 |
|   10424 |       235.2000 |           139 | 11493.2000 |
|   10817 |       362.4300 |           155 | 11490.7000 |
|   10889 |       284.5000 |            80 | 11380.0000 |
|   10417 |       240.8000 |           123 | 11283.2000 |
|   10897 |       149.6800 |           116 | 10835.2400 |
|   10353 |       227.6000 |            62 | 10741.6000 |
+---------+----------------+---------------+------------+
10 rows in set (0,00 sec)
```

12) Faça uma consulta mostrando o ID do pedido (OrderID) e o preço total do pedido. Use

```
mysql>  select OrderID, Sum(UnitPrice*Quantity) as "Preço Total" from OrderDetails group b
+---------+--------------+
| OrderID | Preço Total  |
+---------+--------------+
|   10865 |   17250.0000 |
|   11030 |   16321.9000 |
|   10981 |   15810.0000 |
|   10372 |   12281.2000 |
|   10424 |   11493.2000 |
|   10817 |   11490.7000 |
|   10889 |   11380.0000 |
|   10417 |   11283.2000 |
|   10897 |   10835.2400 |
|   10353 |   10741.6000 |
+---------+--------------+
10 rows in set (0,00 sec)
```

9.1 Exercício de Fixação

Nas consultas pedidas a seguir utilizamos a tabela 'Suppliers'.

1. Informe o ID, o nome da empresa, a cidade, a região e o país de todos os fornecedores salvos no banco de dados.

```
select SupplierID, CompanyName, City,Region,Country from Suppliers;
```

2. Liste todos os países e o número de fornecedores que cada um páis possui. Ordene por ordem descrescente do número de fornecedores.

```
select count(SupplierID),Country from Suppliers
    -> group by Country
    -> order by Count(SupplierID) desc;
```

3. Liste o ID, o nome da empresa, a cidade, a região e o país de todos os fornecedores salvos no banco de dados cuja região informada não está vazia (NULL).

```
select SupplierID, CompanyName, City,Region,Country from Suppliers
    -> where Region is not null;
```

4. Liste o ID, o nome da empresa, a cidade, a região e o país dos países nórdicos.

```
 select SupplierID, CompanyName, City,Region,Country from Suppliers
  where Country in ('Finland','Norway','Denmark','Sweden');
```

10 Usando a Cláusula *Having*

Quando usamos funções de agregação não podemos usar WHERE para testar condições. Nesses casos temos que usar a palavra-chave *HAVING*.

A sintaxe para o uso da cláusula *having* é:

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

Usando a Cláusula HAVING no banco de dados northwind2024

23. Informe o número de clientes de cada país.

```
mysql> select count(CustomerID), Country
    -> from Customers
    -> GROUP BY Country;
+------------------+-------------+
| count(CustomerID) | Country     |
+------------------+-------------+
|                11 | Germany     |
|                 5 | Mexico      |
|                 7 | UK          |
|                 2 | Sweden      |
|                11 | France      |
|                 5 | Spain       |
|                 3 | Canada      |
|                 3 | Argentina   |
|                 2 | Switzerland |
|                 9 | Brazil      |
|                 2 | Austria     |
|                 3 | Italy       |
|                 2 | Portugal    |
|                13 | USA         |
|                 4 | Venezuela   |
|                 1 | Ireland     |
|                 2 | Belgium     |
|                 1 | Norway      |
|                 2 | Denmark     |
|                 2 | NULL        |
|                 2 | Finland     |
|                 1 | Poland      |
+------------------+-------------+
22 rows in set (0,01 sec)
```

24. Agora informe o número de clientes de cada páis considerando apenas os países que possuem mais de 5 clientes.

```
mysql> SELECT COUNT(CustomerID), Country
    -> FROM Customers
    -> GROUP BY Country
    -> HAVING COUNT(CustomerID) > 5;
+------------------+---------+
| COUNT(CustomerID) | Country |
+------------------+---------+
|                11 | Germany |
|                 7 | UK      |
|                11 | France  |
|                 9 | Brazil  |
|                13 | USA     |
+------------------+---------+
```

25. Repita a consulta anterior, agora ordenando pelo número de Clientes decrescente.

```
mysql> SELECT COUNT(CustomerID), Country FROM Customers GROUP BY Country HAVING COUNT
    -> order by Count(CUstomerID) desc;
+-------------------+---------+
| COUNT(CustomerID) | Country |
+-------------------+---------+
|                13 | USA     |
|                11 | Germany |
|                11 | France  |
|                 9 | Brazil  |
|                 7 | UK      |
+-------------------+---------+
5 rows in set (0,00 sec)
```

26. Selecione os IDs e os Preços dos produtos que possuem $100 <$ Preço < 1000

```
mysql> select distinct ProductID, UnitPrice from OrderDetails
 where UnitPrice > 100 and UnitPrice <1000;
+-----------+-----------+
| ProductID | UnitPrice |
+-----------+-----------+
|        38 |  210.8000 |
|        29 |  123.7900 |
|        38 |  263.5000 |
+-----------+-----------+
3 rows in set (0,00 sec)
```

Pergunta: por que usar *DISTINCT* na consulta acima?

27. Repita a consulta anterior agora usando a palavra-chave BETWEEN:

```
mysql> select distinct ProductID, UnitPrice from OrderDetails
where UnitPrice between 100 and 1000;
+-----------+-----------+
| ProductID | UnitPrice |
+-----------+-----------+
|        38 |  210.8000 |
|        29 |  123.7900 |
|        38 |  263.5000 |
+-----------+-----------+
3 rows in set (0,00 sec)
```

28. Informe o ID do pedido, o preço total por unidade, a quantidade total pedida o valor total do pedido (quantidade total $\times$ preço unitário). Ordene pelo valor total maior, decrescente. Mostre apenas os pedidos que possuem o valor total acima de 30000 (trinta mil).

```
mysql>  select OrderID, sum(UnitPrice) as "Preço Unitário Total",
     sum(Quantity) as "Quantidade Total do Pedido",
     sum(UnitPrice)*sum(Quantity) as "Valor Total Pedido"
     from OrderDetails group by OrderID
     having sum(UnitPrice)*sum(Quantity) > 30000
     order by sum(UnitPrice)*sum(Quantity) desc;
+---------+-----------------------+---------------------------+--------------------
| OrderID | Preço Unitário Total  | Quantidade Total do Pedido | Valor Total Pedido
+---------+-----------------------+---------------------------+--------------------
|   11030 |              219.1400 |                       330 |         72316.2000
|   10817 |              362.4300 |                       155 |         56176.6500
|   10515 |              194.8500 |                       286 |         55727.1000
|   10372 |              330.6000 |                       164 |         54218.4000
|   10540 |              317.2300 |                       165 |         52342.9500
|   10360 |              368.1000 |                       138 |         50797.8000
|   10691 |              256.5400 |                       182 |         46690.2800
|   10479 |              307.6000 |                       148 |         45524.8000
|   10514 |              193.4000 |                       233 |         45062.2000
|   11077 |              574.3500 |                        72 |         41353.2000
|   10865 |              281.5000 |                       140 |         39410.0000
|   11021 |              219.0300 |                       168 |         36797.0400
|   10979 |              172.8000 |                       207 |         35769.6000
|   10847 |              118.2000 |                       288 |         34041.6000
|   10424 |              235.2000 |                       139 |         32692.8000
|   10612 |              122.0000 |                       263 |         32086.0000
|   10440 |              150.9000 |                       208 |         31387.2000
|   11032 |              337.5000 |                        90 |         30375.0000
|   10351 |              250.8000 |                       120 |         30096.0000
+---------+-----------------------+---------------------------+--------------------
19 rows in set (0,00 sec)
```

29. Liste todos os empregados (Nome e Sobrenome), seus países e salários.

```
mysql> select FirstName, LastName, Country, Salary  from Employees;
+-----------+-----------+---------+---------+
| FirstName | LastName  | Country | Salary  |
+-----------+-----------+---------+---------+
| Nancy     | Davolio   | USA     | 2954.55 |
| Andrew    | Fuller    | USA     | 2254.49 |
| Janet     | Leverling | USA     | 3119.15 |
| Margaret  | Peacock   | USA     | 1861.08 |
| Steven    | Buchanan  | UK      | 1744.21 |
| Michael   | Suyama    | UK      | 2004.07 |
| Robert    | King      | UK      | 1991.55 |
| Laura     | Callahan  | USA     |  2100.5 |
| Anne      | Dodsworth | UK      | 2333.33 |
+-----------+-----------+---------+---------+
9 rows in set (0,00 sec)
```

30. Liste a média dos salários por país.

```
mysql> select avg(Salary), Country  from Employees
group by Country order by sum(Salary) desc;
+-------------------+---------+
| avg(Salary)       | Country |
+-------------------+---------+
| 2457.9539794921875 | USA     |
| 2018.2900085449219 | UK      |
+-------------------+---------+
2 rows in set (0,00 sec)
```

31. Liste os empregados que possuem salários maiores que 2100.

```
mysql> select FirstName, LastName, Country, Salary  from Employees
    -> where Salary > 2100;
+-----------+-----------+---------+---------+
| FirstName | LastName  | Country | Salary  |
+-----------+-----------+---------+---------+
| Nancy     | Davolio   | USA     | 2954.55 |
| Andrew    | Fuller    | USA     | 2254.49 |
| Janet     | Leverling | USA     | 3119.15 |
| Laura     | Callahan  | USA     |  2100.5 |
| Anne      | Dodsworth | UK      | 2333.33 |
+-----------+-----------+---------+---------+
5 rows in set (0,00 sec)
```

32. Liste o(s) país(es) que possui(em) o total de gasto(s) por salário maior que 10000.

```
mysql> select sum(Salary), Country  from Employees group by Country
    -> having sum(Salary) > 10000
    -> order by sum(Salary) desc;
+-------------------+---------+
| sum(Salary)       | Country |
+-------------------+---------+
| 12289.769897460938 | USA     |
+-------------------+---------+
1 row in set (0,00 sec)
```

Principais Comandos do Mysql

fantini@coltec.ufmg

21 de agosto de 2026

1. Operações nos Bancos de Dados

- **Create Database** (Cria Banco de Dados): **CREATE DATABASE meubanco**
- **Use Database** (Seleciona Banco de Dados): **USE meubanco**
- **Drop Database** (Remove Banco de Dados): **DROP DATABASE meubanco**
- **Show Databases** (Lista todos Banco de Dados): **SHOW DATABASES**

2. Operações em Tabelas

- Create Table (Cria Tabela): `CREATE TABLE tabela1 (id int, nome varchar(40))`
- Drop Table (Remove Tabela): `DROP TABLE tabela1`
- Rename Table (Renomeia Tabela): `RENAME TABLE tabela1 to tabela2`
- Show Tables (Lista Todas Tabelas do Banco de Dados): `SHOW TABLES`
- Describe Table (Descreve a Estrutura da Tabela): `describe tabela1`
- Truncate Table (Apaga todas as linhas de uma tabela): `truncate tabela1`
  - Compare com o comando **DELETE FROM**.

2.1. Usando o Comando ALTER TABLE

- ALTER TABLE...ADD (Adiciona Coluna): `ALTER TABLE tabela1 ADD Coluna10`
- ALTER TABLE...DROP (Remove Coluna): `ALTER TABLE tabela1 DROP Coluna10`
- ALTER TABLE...RENAME COLUMN... (Renomeia Coluna):
```
ALTER TABLE tabela1 RENAME COLUMN ColunaX TO ColunaY
```
  - Comando disponível a partir da versão 8.0. Versões anteriores devem usar o comando *ALTER TABLE ...CHANGE ...* como mostrado a seguir.
- ALTER TABLE...CHANGE... (Renomeia Coluna):
```
ALTER TABLE tabela1 CHANGE ColunaX ColunaY TIPO_DO_DADO
```
  - O tipo de dado (data type) tem que ser informado.
  - Podemos alterar o tipo de dados da coluna ou manter o existente.
  - Em ambos os casos temos que especificá-lo.
  - Para verificar o tipo de dados da coluna usamos o comando *DESCRIBE*.
- ALTER TABLE...MODIFY (Altera o Tipo de Dado da Coluna):
```
ALTER TABLE tabela1 MODIFY Coluna10 TIPO_DE_DADO
```
- ALTER TABLE...ADD PRIMARY KEY ... (Adiciona Chave Primária):
```
ALTER TABLE tabela1 ADD PRIMARY KEY (Coluna1)
```
  - Esse comando modifica a coluna para que o campo se torne chave primária.
- ALTER TABLE...DROP PRIMARY KEY ... (**Remove** Chave Primária):
```
ALTER TABLE tabela1 DROP PRIMARY KEY (Coluna1)
```
  - Esse comando modifica a coluna removendo a especificação ‘‘chave primária’’ de uma coluna (campo) (mas mantendo a coluna (campo)).

- ALTER TABLE...ADD CONSTRAINT ...FOREIGN KEY ...REFERENCES ... (Adiciona Chave Estrangeira):
  ALTER TABLE tabela1 ADD CONSTRAINT nome_fk FOREIGN KEY (Coluna1) REFERENCES tabela2(Coluna1)
  - Esse comando modifica a coluna para que o campo se torne chave estrangeira, fazendo referência a uma coluna de outra tabela.
  - Pelo SQL padrão, a referência de uma chave estrangeira deve ser a PRIMARY KEY ou uma UNIQUE KEY na tabela referenciada.
- ALTER TABLE...DROP FOREIGN KEY ... (**Remove** Chave Estrangeira):
  ALTER TABLE tabela1 DROP FOREIGN KEY (Nome_FK)
  - Esse comando modifica a coluna removendo a especificação ‘‘chave estrangeira’’ (*foreing key*) de uma coluna (mas mantendo a coluna).
- ALTER TABLE...ADD UNIQUE... (Adiciona restrição de unicidade para o campo):
  ALTER TABLE tabela1 ADD UNIQUE (coluna1)
  - Esse comando modifica a coluna adiciona a restrição de que o campo não tenha valores repetidos.

2.2. Índices

- CREATE INDEX...ON (Cria um índice em uma Tabela):
  CREATE INDEX Nome-do-Indice ON tabela1(coluna1)
- DROP INDEX...ON (Remove índice em uma Tabela):
  DROP INDEX Nome-do-Indice ON tabela1

3. Manipulação de Dados

3.1. Insere, Atualiza, Remove

- INSERT INTO ... VALUES ... (Insere Registros (linhas) em uma Tabela):
  INSERT INTO tabela1 (coluna1, coluna2,... colunaN) VALUES (valor1, valor2,...valorN)
  - Caso a inserção de dados seja em **todas colunas** pode ser usar o comando:
    * INSERT INTO tabela1 VALUES (valor1, valor2,...valorN)
- UPDATE ....SET (Atualiza valores dos campos):
  UPDATE tabela1 SET coluna1 = valor1 WHERE condição
- DELETE ... FROM (Remove linhas):
  DELETE FROM tabela1 WHERE condição

3.2. Usando o SELECT

- **SELECT * FROM ...** (Seleciona Dados de uma Tabela): SELECT * FROM tabela1
- **SELECT * FROM ... WHERE ...** (Seleciona Dados com Condições):

  SELECT * FROM tabela1 WHERE Condição

  Condições Lógicas:
- **SELECT coluna1, coluna2 ... FROM tabela WHERE condição AND condição** :
- **SELECT coluna1, coluna2 ... FROM tabela WHERE condição OR condição** :
- **SELECT coluna1, coluna2 ... FROM tabela WHERE NOT condição** :

  Order By
- **SELECT * FROM ... WHERE ... ORDER BY** (Seleciona Dados com Condições e Ordena):

  SELECT * FROM tabela1 WHERE Condição ORDER BY coluna (ASC ou DESC)
- **SELECT DISTINCT coluna FROM ...** (Seleciona Dados Distintos):

  SELECT DISTINCT coluna FROM tabela1
- **SELECT COUNT(*) FROM ...** (Conta número de linhas):

  SELECT COUNT(*) FROM tabela1
- **SELECT SUM(coluna) FROM ...** (Soma dados de uma coluna):

  SELECT SUM(coluna1) FROM tabela1
- **SELECT AVG(coluna) FROM ...** (Calcula a média dos dados de uma coluna):

  SELECT AVG(coluna1) FROM tabela1
- **SELECT coluna1, COUNT(*) FROM ...GROUP BY ...** (Conta linhas de um campo, agrupados por categoria):

  SELECT coluna1, COUNT(*) FROM tabela1 GROUP BY coluna1
- **SELECT coluna1, coluna2, COUNT(*) FROM ...GROUP BY ...HAVING ...** (Usa a condição having para filtrar a consulta):

  SELECT coluna1, coluna2, COUNT(*) FROM tabela1 GROUP BY coluna1, coluna2
  HAVING COUNT(*) > 1

  Ordem dos Comandos

  - SELECT...
  - FROM...
  - WHERE...
  - GROUP BY...
  - HAVING...
  - ORDER BY...
  - LIMIT

4. Junções (Join)

- INNER JOIN (Retorna apenas as linhas que têm correspondência em ambas as tabelas):
  ```
  SELECT * FROM tabela1 INNER JOIN tabela2
  ON tabela1.coluna1 = tabela2.coluna2
  ```
  - É o tipo de junção mais comum. Se uma linha de tabela1 não tem correspondente em tabela2 (ou vice-versa), ela **não** aparece no resultado.
- LEFT JOIN (Retorna todas as linhas da tabela da esquerda, mesmo sem correspondência na direita):
  ```
  SELECT * FROM tabela1 LEFT JOIN tabela2
  ON tabela1.coluna1 = tabela2.coluna2
  ```
  - Quando não há correspondência em tabela2, as colunas dela aparecem como NULL.
- RIGHT JOIN (Retorna todas as linhas da tabela da direita, mesmo sem correspondência na esquerda):
  ```
  SELECT * FROM tabela1 RIGHT JOIN tabela2
  ON tabela1.coluna1 = tabela2.coluna2
  ```
  - O espelho do LEFT JOIN -- na prática, quase tudo que se faz com RIGHT JOIN também dá para fazer com LEFT JOIN só invertendo a ordem das tabelas. A maioria dos programadores prefere usar sempre LEFT JOIN por consistência.
- CROSS JOIN (Produto cartesiano -- combina cada linha de uma tabela com cada linha da outra):
  ```
  SELECT * FROM tabela1 CROSS JOIN tabela2
  ```
  - Não usa ON -- não há condição de correspondência. Se tabela1 tem 10 linhas e tabela2 tem 5, o resultado tem 50 linhas. Uso raro no dia a dia; útil para gerar combinações (ex.: todos os pares possíveis de produto e cor).
- FULL OUTER JOIN (Retorna todas as linhas de ambas as tabelas, com ou sem correspondência):
  - O MySQL **não tem** o comando FULL OUTER JOIN diretamente (diferente de PostgreSQL e SQL Server). Para obter o mesmo resultado, combina-se um LEFT JOIN e um RIGHT JOIN com UNION:
    ```
    SELECT * FROM tabela1 LEFT JOIN tabela2 ON tabela1.coluna1 =
    tabela2.coluna2
    UNION
    SELECT * FROM tabela1 RIGHT JOIN tabela2 ON tabela1.coluna1 =
    tabela2.coluna2
    ```

5. Subconsultas (Subqueries)

- **Subconsulta no WHERE, com IN** (Filtra usando o resultado de outra consulta):

```
SELECT * FROM funcionarios
WHERE departamento_id IN
(SELECT id FROM departamentos WHERE cidade = 'Belo Horizonte')
```

  - A subconsulta (entre parênteses) roda primeiro e devolve uma lista de valores, que a consulta externa usa no IN.

- **Subconsulta com EXISTS** (Verifica se a subconsulta retorna alguma linha):

```
SELECT * FROM departamentos d
WHERE EXISTS
(SELECT 1 FROM funcionarios f WHERE f.departamento_id = d.id)
```

  - Retorna os departamentos que têm pelo menos um funcionário. EXISTS costuma ser mais eficiente que IN quando a subconsulta pode retornar muitas linhas.

- **Subconsulta no FROM** (Usa o resultado de uma consulta como se fosse uma tabela):

```
SELECT departamento_id, media_salario FROM
(SELECT departamento_id, AVG(salario) AS media_salario
FROM funcionarios GROUP BY departamento_id) AS resumo
WHERE media_salario > 5000
```

  - A subconsulta precisa de um apelido (AS resumo) para ser referenciada.

6. Funções Comumente Usadas

6.1. Funções de Texto (String)

- CONCAT (Junta textos): `SELECT CONCAT(nome, ' - ', cargo) FROM funcionarios`
- UPPER / LOWER (Maiúsculas / minúsculas): `SELECT UPPER(nome) FROM funcionarios`
- LENGTH (Tamanho do texto, em caracteres): `SELECT LENGTH(nome) FROM funcionarios`
- SUBSTRING (Extrai parte de um texto): `SELECT SUBSTRING(nome, 1, 3) FROM funcionarios`
  - Nesse exemplo, extrai os 3 primeiros caracteres de nome.
- TRIM (Remove espaços do início e do fim): `SELECT TRIM(nome) FROM funcionarios`
- REPLACE (Substitui um trecho de texto por outro): `SELECT REPLACE(telefone, '-', '') FROM funcionarios`

6.2. Funções de Data e Hora

- NOW() (Data e hora atuais): `SELECT NOW()`
- CURDATE() (Apenas a data atual): `SELECT CURDATE()`
- DATE_FORMAT (Formata uma data): `SELECT DATE_FORMAT(data_contratacao, '%d/%m/%Y') FROM funcionarios`
- DATEDIFF (Diferença, em dias, entre duas datas): `SELECT DATEDIFF(NOW(), data_contratacao) FROM funcionarios`
- DATE_ADD (Soma um intervalo a uma data): `SELECT DATE_ADD(data_contratacao, INTERVAL 1 YEAR) FROM funcionarios`

6.3. Funções Numéricas

- ROUND (Arredonda um número): `SELECT ROUND(salario, 2) FROM funcionarios`
- CEIL / FLOOR (Arredonda para cima / para baixo): `SELECT CEIL(salario), FLOOR(salario) FROM funcionarios`
- ABS (Valor absoluto): `SELECT ABS(-15)`

7. Tipos de Dados Comuns

Usados em CREATE TABLE e em ALTER TABLE ... MODIFY (Seção 2.1):

Tipo	Uso
INT	Números inteiros
DECIMAL(m,d)	Números decimais exatos -- m dígitos no total, d depois da vírgula. Preferível a FLOAT para dinheiro, por não ter erro de arredondamento.
VARCHAR(n)	Texto de tamanho variável, até n caracteres
TEXT	Texto longo, sem limite prático de tamanho
DATE	Data (ano-mês-dia)
DATETIME	Data e hora
BOOLEAN	Verdadeiro/falso (internamente, um TINYINT)

- NOT NULL (Obriga o campo a ter um valor): `nome VARCHAR(100) NOT NULL`
- DEFAULT (Define um valor padrão quando nenhum é informado): `ativo BOOLEAN DEFAULT TRUE`
- AUTO_INCREMENT (Gera um valor numérico crescente automaticamente -- comum em chaves primárias): `id INT AUTO_INCREMENT PRIMARY KEY`

8. Transações

- START TRANSACTION (Inicia um bloco de operações que serão confirmadas ou desfeitas em conjunto): **START TRANSACTION**
- COMMIT (Confirma definitivamente as alterações da transação): **COMMIT**
- ROLLBACK (Desfaz todas as alterações da transação atual): **ROLLBACK**
- SAVEPOINT (Marca um ponto intermediário para desfazer parcialmente): **SAVEPOINT ponto1**
  - Depois, é possível voltar só até esse ponto com **ROLLBACK TO ponto1**, sem desfazer a transação inteira.

Exemplo de uso: transferir saldo entre duas contas exige que duas operações (debitar de uma, creditar em outra) aconteçam juntas – se uma falhar, a outra não pode ficar "pela metade". Isso é o motivo de existir transação.

9. Usuários e Permissões

- **CREATE USER** (Cria um novo usuário do MySQL):
  `CREATE USER 'nomeusuario'@'localhost' IDENTIFIED BY 'senha'`
  - `'localhost'` restringe de onde o usuário pode se conectar. Usar `'%'` no lugar de `'localhost'` permite conexão de qualquer endereço -- use com cuidado, principalmente em servidores expostos à internet.
- **GRANT** (Concede permissões a um usuário):
  `GRANT SELECT, INSERT, UPDATE ON meubanco.* TO 'nomeusuario'@'localhost'`
  - `meubanco.*` concede a permissão em todas as tabelas do banco. Pode-se restringir a uma tabela específica: `meubanco.tabela1`.
  - `GRANT ALL PRIVILEGES ON meubanco.* TO 'nomeusuario'@'localhost'` concede todas as permissões -- reserve para quem realmente precisa administrar o banco.
- **REVOKE** (Remove permissões de um usuário):
  `REVOKE INSERT ON meubanco.* FROM 'nomeusuario'@'localhost'`
- **FLUSH PRIVILEGES** (Recarrega as permissões, garantindo que a mudança tenha efeito imediato): `FLUSH PRIVILEGES`
- **SHOW GRANTS** (Lista as permissões de um usuário): `SHOW GRANTS FOR 'nomeusuario'@'localhost'`
- **DROP USER** (Remove um usuário): `DROP USER 'nomeusuario'@'localhost'`

10. Views

- **CREATE VIEW** (Cria uma "tabela virtual", definida por uma consulta):
  ```
  CREATE VIEW funcionarios_bh AS
  SELECT * FROM funcionarios WHERE cidade = 'Belo Horizonte'
  ```
  - Depois de criada, a view se usa como uma tabela comum: `SELECT * FROM funcionarios_bh`.
  - Não guarda os dados fisicamente -- a consulta é executada de novo toda vez que a view é usada.
- **DROP VIEW** (Remove uma view): `DROP VIEW funcionarios_bh`

11. Exemplos

Esta seção reúne os comandos das seções anteriores em um estudo de caso único e coerente, do zero: criação das tabelas, inserção de dados, e consultas cada vez mais elaboradas sobre elas.

11.1. Criando o Banco e as Tabelas

```
CREATE DATABASE empresa

USE empresa

CREATE TABLE departamentos (
id INT AUTO_INCREMENT PRIMARY KEY,
nome VARCHAR(50) NOT NULL,
cidade VARCHAR(50)
)

CREATE TABLE funcionarios (
id INT AUTO_INCREMENT PRIMARY KEY,
nome VARCHAR(100) NOT NULL,
cargo VARCHAR(50),
salario DECIMAL(10,2),
data_contratacao DATE,
departamento_id INT,
FOREIGN KEY (departamento_id) REFERENCES departamentos(id)
)
```

11.2. Inserindo Dados

```
INSERT INTO departamentos (nome, cidade) VALUES
('TI', 'Belo Horizonte'),
('Vendas', 'São Paulo'),
('RH', 'Belo Horizonte')

INSERT INTO funcionarios (nome, cargo, salario, data_contratacao, departamento_id)
VALUES
('Ana Souza', 'Desenvolvedora', 6500.00, '2022-03-10', 1),
('Bruno Lima', 'Analista', 4200.00, '2023-06-01', 1),
('Carla Dias', 'Vendedora', 3800.00, '2021-11-20', 2),
('Diego Alves', 'Gerente de RH', 7200.00, '2020-01-15', 3)
```

11.3. Consultas

```
Exemplo 1 -- Funcionários e o nome do seu departamento:
SELECT f.nome, f.cargo, d.nome AS departamento
FROM funcionarios f
INNER JOIN departamentos d ON f.departamento_id = d.id

Exemplo 2 -- Salário médio por departamento, do maior para o menor:
SELECT d.nome, AVG(f.salario) AS media_salario
FROM funcionarios f
INNER JOIN departamentos d ON f.departamento_id = d.id
```

```
GROUP BY d.nome
ORDER BY media_salario DESC

Exemplo 3 -- Departamentos que ainda não têm nenhum funcionário (LEFT JOIN):
SELECT d.nome
FROM departamentos d
LEFT JOIN funcionarios f ON f.departamento_id = d.id
WHERE f.id IS NULL

Exemplo 4 -- Funcionários que ganham acima da média geral (subconsulta):
SELECT nome, salario FROM funcionarios
WHERE salario > (SELECT AVG(salario) FROM funcionarios)

Exemplo 5 -- Tempo de empresa de cada funcionário, em dias:
SELECT nome, DATEDIFF(NOW(), data_contratacao) AS dias_de_empresa
FROM funcionarios
ORDER BY dias_de_empresa DESC

Exemplo 6 -- Criando um usuário só de leitura para este banco:
CREATE USER 'leitor'@'localhost' IDENTIFIED BY 'senhaSegura123'
GRANT SELECT ON empresa.* TO 'leitor'@'localhost'
FLUSH PRIVILEGES
```